

**HO CHI MINH CITY UNIVERSITY OF TRANSPORT**

**FACULTY OF INFORMATION TECHNOLOGY**



**CAPSTONE PROJECT REPORT**

**COURSE: SOFTWARE ENGINEERING**

**PERSONALIZED ADAPTIVE LEARNING  
PLATFORM WITH INTEGRATED AI TUTOR**

**Project Members:**

Tran Duc Hai — 036207010989 (Leader)

Nguyen Anh Huy — 066207008839

Luu Minh Hieu — 079201009126

Nguyen Huu Tri — 079207046324

Nguyen Van Minh — 064205001508

**Academic Advisor / Lecturer:**

M.Sc. Nguyen Van Chien

**Ho Chi Minh City, September 2026**

# Contents

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>Definitions and Acronyms (Glossary)</b>                                    | <b>1</b>  |
| <b>1 Project Introduction</b>                                                 | <b>4</b>  |
| 1.1 Overview                                                                  | 4         |
| 1.1.1 Project Information                                                     | 4         |
| 1.1.2 Project Team & Responsibilities                                         | 4         |
| 1.2 Product Background & Problem Statement                                    | 4         |
| 1.2.1 Background                                                              | 4         |
| 1.2.2 Problem Statement                                                       | 4         |
| 1.3 Existing Systems Comparison                                               | 5         |
| <b>2 Project Management Plan</b>                                              | <b>5</b>  |
| 2.1 Overview & Scope Estimation (WBS & Effort Estimation)                     | 5         |
| 2.2 Project Objectives & Quality Metrics                                      | 6         |
| 2.3 Project Risks & Response Plans                                            | 7         |
| 2.4 Management Approach & Process (Agile/Scrum 8-Week Timeline)               | 7         |
| 2.5 Project Communications & Configuration Management                         | 8         |
| 2.5.1 Communication Channels                                                  | 8         |
| 2.5.2 Configuration & Version Control Strategy                                | 8         |
| <b>3 Software Requirement Specification (SRS)</b>                             | <b>8</b>  |
| 3.1 User Requirements                                                         | 8         |
| 3.1.1 System Actors Analysis                                                  | 8         |
| 3.2 Functional Requirements (Core Business Flows)                             | 10        |
| 3.2.1 Core Flow 1 (UC-01): Competency Diagnosis & Learning Profile Generation | 10        |
| 3.2.2 Core Flow 2 (UC-02): AI Roadmap Generation                              | 12        |
| 3.2.3 Core Flow 3 (UC-03): Real-Time Adaptive Learning Loop                   | 13        |
| 3.2.4 Core Flow 4 (UC-04): AI Tutor Support with RAG                          | 14        |
| 3.2.5 Core Flow 5 (UC-05): Learning Analytics and Progress Monitoring         | 15        |
| 3.2.6 Core Flow 6 (UC-06): Examination Performance Prediction                 | 16        |
| 3.3 Non-Functional Requirements (NFR)                                         | 17        |
| <b>4 Software Design Description (SDD)</b>                                    | <b>18</b> |
| 4.1 System Architecture Design                                                | 18        |
| 4.1.1 High-Level System Architecture                                          | 18        |
| 4.1.2 Package Diagram                                                         | 19        |
| 4.2 Use Case Diagrams Design                                                  | 19        |

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| 4.2.1    | Overall System Use Case Diagram . . . . .                                 | 19        |
| 4.2.2    | Student & AI Interactive Use Case Diagram . . . . .                       | 20        |
| 4.2.3    | Teacher & Admin Use Case Diagram . . . . .                                | 21        |
| 4.2.4    | Parent Use Case Diagram . . . . .                                         | 22        |
| 4.3      | Database Design (ERD & Vector DB) . . . . .                               | 23        |
| 4.3.1    | Relational Database Schema (ERD) . . . . .                                | 23        |
| 4.3.2    | Vector Database Architecture & AI RAG Pipeline . . . . .                  | 23        |
| 4.3.3    | AI RAG Processing Algorithms . . . . .                                    | 24        |
| 4.4      | Sequence Diagrams & Detailed Design . . . . .                             | 24        |
| <b>5</b> | <b>System Implementation &amp; Testing</b>                                | <b>26</b> |
| 5.1      | Implementation Tech Stack . . . . .                                       | 26        |
| 5.2      | System Features & User Interface Walkthrough . . . . .                    | 27        |
| 5.2.1    | Authentication & Multi-Role Access Control (RBAC) . . . . .               | 27        |
| 5.2.2    | Competency Diagnosis & Baseline Assessment Module (UC-01) . . . . .       | 28        |
| 5.2.3    | Goal Setting & Personalized Learning Roadmap Module (UC-02) . . . . .     | 30        |
| 5.2.4    | Topic-Based Adaptive Practice Module (UC-03) . . . . .                    | 30        |
| 5.2.5    | Intelligent AI Tutor with RAG Integration (UC-04) . . . . .               | 32        |
| 5.2.6    | Student Overview & Competency Progress Analytics Module (UC-05) . . . . . | 33        |
| 5.2.7    | Teacher Portal: Question Bank Management System (CRUD) (UC-06) . . . . .  | 35        |
| 5.2.8    | Teacher Portal: Class-Wide Performance Analytics (UC-07) . . . . .        | 37        |
| 5.2.9    | Parent Portal Subsystem . . . . .                                         | 37        |
| 5.2.10   | Teacher Overview & Knowledge Base (RAG) . . . . .                         | 39        |
| 5.3      | Automated Unit & Integration Testing Results . . . . .                    | 41        |

# Definitions and Acronyms (Glossary)

## 1. Actor & Access Control Terms

| Term           | Full Name                  | Description                                                                                                                                         |
|----------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor          | System Actor               | An entity interacting with the system to achieve a specific business goal. Main actors: Administrator, Teacher / Academic Manager, Student, Parent. |
| Administrator  | System Administrator       | Responsible for overall system management, user accounts, permissions, AI service configuration, and monitoring.                                    |
| Teacher        | Teacher / Academic Manager | Responsible for building learning materials, managing question banks, monitoring student results, and reviewing AI-generated content.               |
| Student        | Student                    | Primary user who completes diagnostic assessments, learns via personalized roadmaps, practices adaptively, and interacts with AI Tutor.             |
| Parent         | Parent                     | Monitors student learning progress, evaluation results, and receives notifications in read-only mode via Parent Portal.                             |
| RBAC           | Role-Based Access Control  | Access control mechanism based on roles to ensure users access authorized data and functions only.                                                  |
| Authorization  | User Authorization         | Process of checking user access rights after authentication to determine the functions permitted for use within the system.                         |
| Authentication | User Authentication        | Process of verifying user identity before allowing access to the system through a login account.                                                    |
| JWT            | JSON Web Token             | Standardized token format used for authenticating user requests securely without re-login.                                                          |

## 2. Mathematical & AI Metrics

| Term             | Full Name                           | Description                                                                                                                            |
|------------------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| $\theta$ (Theta) | Student Ability / Proficiency Level | Real-time ability measure of a student ( $[-3, +3]$ or $[0, 1]$ ), representing mastery over a topic.                                  |
| $b$              | Question Difficulty Level           | Standard difficulty level of a test question. Higher values indicate higher complexity.                                                |
| $\Delta\theta$   | Ability Modulation / Delta          | The increase/decrease in ability index immediately following a correct or incorrect answer.                                            |
| $\pm\Delta b$    | Difficulty Tolerance Range          | Allowed difficulty variance when an exact difficulty match $b$ is missing in the database.                                             |
| IRT              | Item Response Theory                | Mathematical model measuring the probability of a correct response given student ability ( $\theta$ ) and question difficulty ( $b$ ). |
| BKT              | Bayesian Knowledge Tracing          | Algorithmic model tracing student understanding probability across sequential exercise attempts.                                       |
| ZPD              | Zone of Proximal Development        | Optimal difficulty range matching student current ability—neither too easy nor overly frustrating.                                     |

### 3. AI & Technical Terms

| Term              | Full Name                      | Description                                                                                                       |
|-------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------|
| RAG               | Retrieval-Augmented Generation | Combines external vector DB knowledge retrieval with LLM prompts to produce accurate, hallucination-free answers. |
| LLM               | Large Language Model           | Large language model (OpenAI GPT, Gemini, Anthropic) serving as the natural language engine for AI Tutor.         |
| Vector DB         | Vector Database                | Specialized database (Milvus/Pinecone) storing vector embeddings for high-speed semantic similarity searches.     |
| Cosine Similarity | Cosine Similarity Search       | Mathematical measure comparing semantic similarity between student prompts and context vectors in Vector DB.      |
| SSE               | Server-Sent Events             | One-way real-time streaming protocol from Server to Client for smooth word-by-word chatbot output.                |
| TTFT              | Time To First Token            | Latency from user prompt submission until the first response token is displayed (< 1.5s).                         |

# 1 Project Introduction

## 1.1 Overview

### 1.1.1 Project Information

- **Project Title:** Personalized Adaptive Learning Platform with Integrated AI Tutor
- **Domain:** Educational Technology (EdTech), Adaptive Learning Systems, Artificial Intelligence in Education.
- **Primary Target:** National Competency Assessment Examination preparation and standardized academic evaluation.

### 1.1.2 Project Team & Responsibilities

| Full Name        | Role     | Email                  | Phone      |
|------------------|----------|------------------------|------------|
| Nguyen Van Chien | Lecturer | chiennv@ut.edu.vn      | 0986964972 |
| Tran Duc Hai     | Leader   | haitd360989@ut.edu.vn  | 0325932899 |
| Nguyen Anh Huy   | Member   | huyna668839@ut.edu.vn  | 0328940225 |
| Nguyen Huu Tri   | Member   | trinh796324@ut.edu.vn  | 0982290149 |
| Nguyen Van Minh  | Member   | minhmv1508@ut.edu.vn   | 0342324611 |
| Luu Minh Hieu    | Member   | hieulm799126@ut.edu.vn | 0334466229 |

## 1.2 Product Background & Problem Statement

### 1.2.1 Background

In recent years, standardized competency exams have become a crucial benchmark for higher education admissions. However, students preparing for these exams face significant structural challenges in their study routines. Traditional test preparation relies heavily on a “one-size-fits-all” model, where every student follows an identical syllabus regardless of their individual baseline knowledge, strengths, or weaknesses.

### 1.2.2 Problem Statement

- **Lack of Real-Time Ability Measurement:** Traditional e-learning platforms measure performance using static percentage scores rather than estimating true latent ability ( $\theta$ ), making it difficult to pinpoint exact knowledge gaps.

- **Generic and Hallucination-Prone AI Assistance:** Standard conversational AI models often deliver hallucinated or unverified solutions that do not strictly adhere to verified curriculum standards.
- **Inflexible Learning Paths & High Costs:** Private tutoring and offline test prep centers present high financial barriers while failing to offer dynamic, continuous adjustments based on daily student effort and cognitive overload.

### 1.3 Existing Systems Comparison

| Criteria           | Traditional LMS & Offline Centers   | Generic AI Chatbot Platforms    | Proposed Platform                                       |
|--------------------|-------------------------------------|---------------------------------|---------------------------------------------------------|
| Assessment Model   | Fixed question sets, static scoring | Unstructured prompts            | Dynamic Item Response Theory (IRT) ( $\theta$ & $b$ )   |
| Learning Path      | Linear, manual progression          | None / Static text suggestion   | Automated AI-driven Roadmap with dynamic recalibration  |
| AI Answer Accuracy | Not available                       | Susceptible to AI Hallucination | High accuracy via RAG Engine & Vector DB Knowledge Base |
| Progress Tracking  | Basic score logs                    | None                            | Multi-actor Analytics (Student, Teacher, Parent)        |

## 2 Project Management Plan

### 2.1 Overview & Scope Estimation (WBS & Effort Estimation)

The project utilizes a Work Breakdown Structure (WBS) to divide total engineering effort across 4 main functional streams: Frontend/Mobile Development, Backend Infrastructure, AI/RAG Engineering, and Quality Assurance/Documentation.

**Effort Distribution (Total Estimated Effort: 600 Person-Hours / 8 Weeks)**

| Work Package / Task Group                       | Primary Responsibility                                       | Assigned Members        | Estimated Effort        |
|-------------------------------------------------|--------------------------------------------------------------|-------------------------|-------------------------|
| WP1: Requirement Analysis & System Architecture | System Architecture & Database Design                        | Leader & All Members    | 80 Person-Hours         |
| WP2: AI & Adaptive Engine Integration           | IRT Algorithm, RAG Pipeline, Vector DB Setup                 | AI Engineers            | 180 Person-Hours        |
| WP3: Core Service Backend API                   | RESTful APIs, JWT Auth, RBAC, Database Management            | Backend Engineers       | 160 Person-Hours        |
| WP4: User Interface & Analytics                 | Web/Mobile Application (Student, Teacher, Parent Dashboards) | Frontend Engineers      | 120 Person-Hours        |
| WP5: Testing, Deployment & Documentation        | Unit/Integration Tests, SRS/SDD Docs, Cloud Setup            | QA & Documentation Lead | 60 Person-Hours         |
| <b>Total Estimated Effort (8 Weeks)</b>         |                                                              |                         | <b>600 Person-Hours</b> |

## 2.2 Project Objectives & Quality Metrics

To ensure the system meets academic and technical expectations, the team enforces key quantitative quality metrics across the development lifecycle:

- **Functional Completeness:** 100% implementation of defined Core Business Flows (UC-01 to UC-06).
- **Performance Benchmark:**
  - API Response Latency < 3.0 seconds for standard CRUD operations.
  - AI Tutor Time-To-First-Token (TTFT) < 1.5 seconds utilizing Server-Sent Events (SSE) streaming.
- **AI Output Accuracy:** Hallucination rate strictly maintained < 2.0% by enforcing RAG context retrieval with Cosine Similarity thresholds (Score  $\geq 0.65$ ).
- **Security & Authorization:** 0 unauthorized data leakages across roles (RBAC) enforced with JWT authentication and Prompt Sanitization against Prompt Injection attacks.

## 2.3 Project Risks & Response Plans

| Risk ID | Risk Category        | Risk Description                                                              | Probability / Impact | Mitigation & Response Plan                                                                                                 |
|---------|----------------------|-------------------------------------------------------------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------|
| RSK-01  | Technical (AI)       | High LLM API latency or Service Rate Limit exceeding quota.                   | Medium / High        | Implement Redis caching for frequent prompts; fallback to static teacher-approved solutions if LLM times out (> 5s).       |
| RSK-02  | Technical (Accuracy) | AI Hallucination or generating answers outside domain scope.                  | Medium / High        | Enforce strict RAG Cosine Similarity filters; apply Guardrail rules; route low-confidence queries to Teacher Ticket queue. |
| RSK-03  | Schedule             | Scope creep or delayed API integration between Frontend and AI Backend.       | High / Medium        | Adopt strict Agile/Scrum weekly sprints; conduct daily standups; maintain continuous integration (CI/CD).                  |
| RSK-04  | Data                 | Cold-start problem for Item Response Theory (IRT) difficulty calibration (b). | High / Medium        | Pre-calibrate initial question difficulty matrices based on historical exam standards and Bloom's Taxonomy tags.           |

## 2.4 Management Approach & Process (Agile/Scrum 8-Week Timeline)

The project follows an Agile/Scrum Framework over an 8-week delivery timeline, divided into 4 two-week Sprints:

- **Week 1–2 (Sprint 1):** Requirements Analysis, SRS Documentation, Architecture & Database Design.
- **Week 3–4 (Sprint 2):** Core Backend Services, IRT Engine Development, Initial Vector DB Setup.
- **Week 5–6 (Sprint 3):** RAG Pipeline Integration, AI Tutor Chatbot, Frontend/Mobile UI Build.
- **Week 7–8 (Sprint 4):** Multi-actor Analytics Dashboards, End-to-End Testing, Bug Fixing, Final Release Package.

- **Sprint Meetings:** Weekly Planning on Mondays; Retrospectives on Fridays.
- **Daily Sync:** 15-minute daily standups via Discord/Zalo to track progress and clear technical blockers.

## 2.5 Project Communications & Configuration Management

### 2.5.1 Communication Channels

- **Source Code Repository:** GitHub / GitLab (Main branch protection enforced; Merge Requests require at least 1 peer review).
- **Task & Progress Management:** Trello / Jira board tracking Backlog, In Progress, Review, and Done.
- **Team Workspace:** Discord for technical discussions; Google Drive for Capstone Documentation.

### 2.5.2 Configuration & Version Control Strategy

1. **Git Branching Strategy:** GitFlow model (main for release, develop for integration, feature/\* for individual task development).
2. **API Documentation:** Postman Collection / Swagger UI for frontend-backend API integration standards.
3. **Environment Configuration:** Separate .env files for Local Development, Staging, and Production (Azure Cloud) to protect sensitive credentials (JWT Secrets, AI API Keys).

## 3 Software Requirement Specification (SRS)

### 3.1 User Requirements

#### 3.1.1 System Actors Analysis

##### System Administrator

- **Goal:** Ensure operational stability, data security, and performance; optimize cloud and AI infrastructure costs; manage user accounts and system configuration.
- **Scope:** Admin Portal, Backend infrastructure, Cloud/Server environments, API Gateways, Audit Logs, and Analytics.
- **Permissions:** CRUD operations on all user accounts (Teacher, Student, Parent); AI/RAG engine parameter configurations; Subscription and Payment package configuration; System health and token usage monitoring.

### **Teacher / Academic Manager**

- **Goal:** Build standardized question banks aligned with Competency Examinations; review AI-generated content (Human-in-the-loop); monitor class and student progress.
- **Scope:** Teacher Portal, Question Bank & Knowledge Base (RAG Vector DB), Class Analytics Dashboard.
- **Permissions:** CRUD questions/exams; upload theory documents into Knowledge Base; review/edit AI-generated explanations; view class analytics and export reports.

### **Student**

- **Goal:** Prepare efficiently for exams via personalized roadmaps; receive 24/7 AI Tutor assistance; track daily progress and achieve target scores.
- **Scope:** Student Web/Mobile App, AI Chatbot interface, Personal Dashboard, Learning Roadmap.
- **Permissions:** Update personal targets and study time; interact with AI Tutor; view competency radar charts, learning streak, and predicted scores.

### **Parent**

- **Goal:** Monitor student learning consistency, performance, and score predictions without interfering with learning workflows.
- **Scope:** Parent Portal / Mobile App (Read-Only Mode).
- **Permissions:** Read-only access to study time, practice test scores, AI predicted scores, and automated alert notifications (absence/drop in activity).

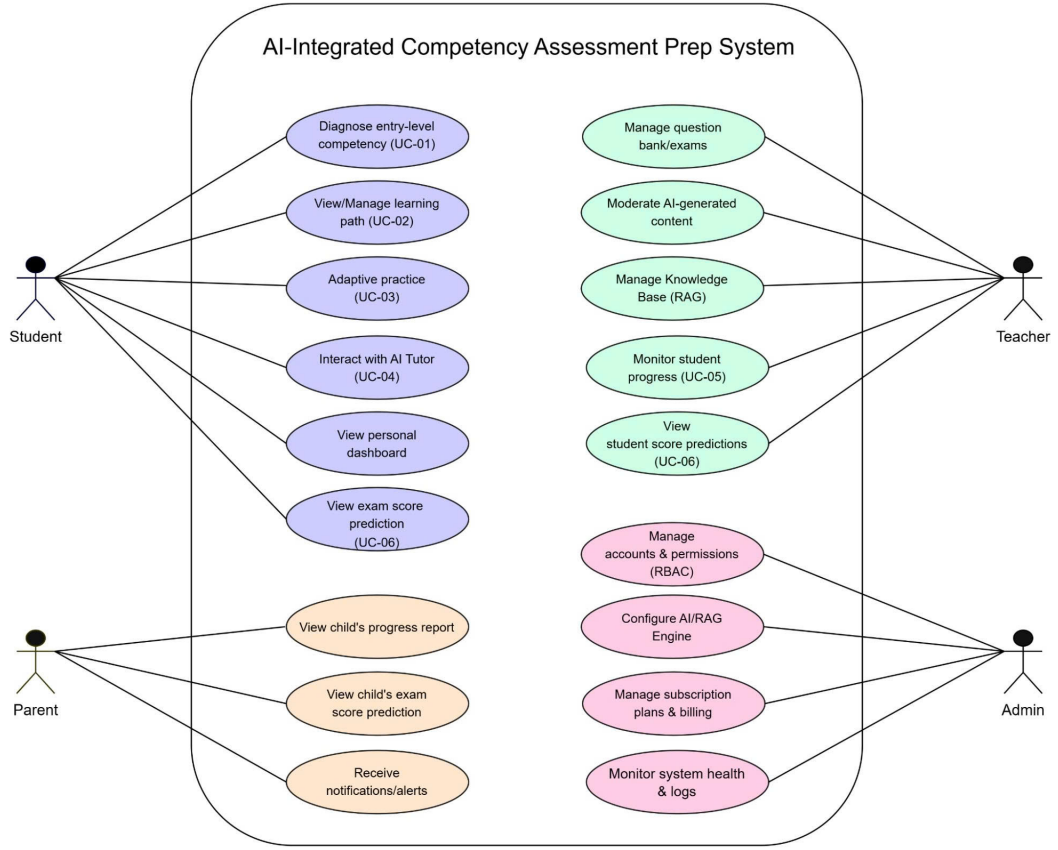


Figure 1: Overall System Use Case Diagram

## 3.2 Functional Requirements (Core Business Flows)

### 3.2.1 Core Flow 1 (UC-01): Competency Diagnosis & Learning Profile Generation

**Actors:** Student (Primary), Assessment Engine, AI Diagnosis Engine, Learning Database.

**Pre-conditions:** Student authenticated via JWT Token; Question Bank approved by Teacher; AI Diagnosis Engine online.

**Basic Flow:**

1. Student initiates a Diagnostic Session via the dashboard. System validates the JWT Token and checks if an existing Learning Profile exists.
2. Student inputs Target Score ( $S_{\text{target}}$ ), Exam Date ( $D_{\text{exam}}$ ), and Daily Study Time ( $T_{\text{daily}}$ ).
3. Assessment Engine queries Question Bank and renders standardized diagnostic test.
4. Student completes and submits test; system logs response choices, time per question, and behavioral indicators.
5. Assessment Engine calculates raw scores and normalizes data.

6. AI Diagnosis Engine computes initial Proficiency Level ( $\theta$ ), identifies Strengths, Weaknesses, Knowledge Gaps, and Learning Preferences.
7. System constructs the Learning Profile data structure, persists it into Learning Database along with Assessment Result history, and associates it with the Student account.
8. System renders the visual diagnostic feedback on the Student Dashboard, including Competency Levels, Knowledge Gap breakdown, Competency Radar Chart, and initial score projections.
9. System finalizes UC-01 and forwards the generated Learning Profile data to Core Flow 2 (UC-02: AI Roadmap Generation).

#### **Alternative Flows:**

- **AF-1.1 Re-assessment:** Student triggers re-evaluation; system generates a new test variant and updates profile history.
- **AF-1.2 Session Interruption:** If Student disconnects or closes the browser mid-test, System persists the current session state and allows resumption upon re-login.
- **AF-1.3 AI Timeout:** If AI Diagnosis times out ( $> 5s$ ), system saves raw results, sets state to “Pending Analysis”, and notifies student.
- **AF-1.4 Insufficient Submission Data:** If the submitted test has excessive skipped questions and lacks minimum data for evaluation, System rejects profile generation and prompts Student to retake the test.

#### **Post-conditions:**

- **Success:** Assessment result is recorded, Learning Profile is generated and persisted in the database, diagnostic feedback is displayed on Student Dashboard, and data is passed to UC-02.
- **Failure:** Learning Profile is not created; session state is saved or system error is logged.

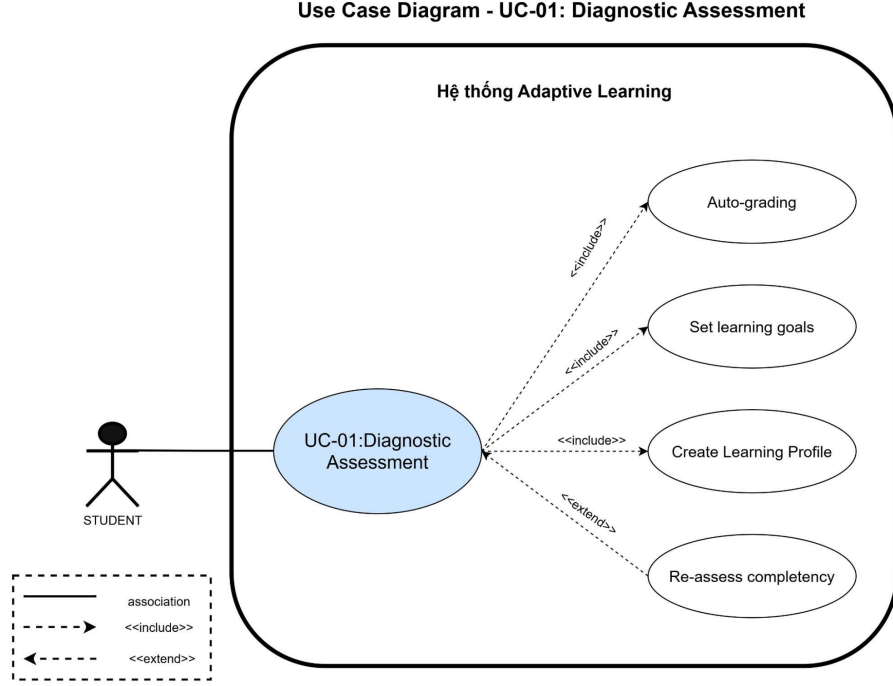


Figure 2: Use Case Diagram — UC-01: Diagnostic Assessment

### 3.2.2 Core Flow 2 (UC-02): AI Roadmap Generation

**Actors:** Student (Primary), AI Planner Engine, Learning Database.

**Pre-conditions:**

- Student is authenticated and possesses a valid Learning Profile generated from UC-01.
- Target learning goals (Target Score  $S_{\text{target}}$  from 1 to 1200, Exam Date  $D_{\text{exam}}$ , Daily Study Time  $T_{\text{daily}}$ ) are defined.
- AI Planner Engine and Learning Database are online and operational.

**Basic Flow:**

1. Triggered automatically after UC-01 or manually via goal updates.
2. AI Planner retrieves latest Learning Profile and Exam Matrix.
3. Gap Analysis is performed against  $S_{\text{target}}$  and remaining days  $N_{\text{days}}$ .
4. Knowledge Graph prioritizes missing prerequisite topics ( $\theta < 0.5$ ).
5. AI constructs phases (Foundation → Topic Mastery → Practice → Final Sprint) and generates structured JSON Daily Tasks aligned with  $T_{\text{daily}}$ .
6. System saves Roadmap in DB and renders interactive timeline dashboard.

### Alternative Flow:

- **AF-2.1 Unrealistic Goal:** If required effort exceeds  $N_{\text{days}} \times T_{\text{daily}}$ , system displays a warning and prompts student to increase daily time or lower target score.

### Post-conditions:

- **Success:** Personalized Study Plan and Daily Tasks are persisted in the database; interactive roadmap timeline is rendered on Student Dashboard.
- **Failure:** Study Plan is not generated; feasibility alert or system error message is displayed to the student.

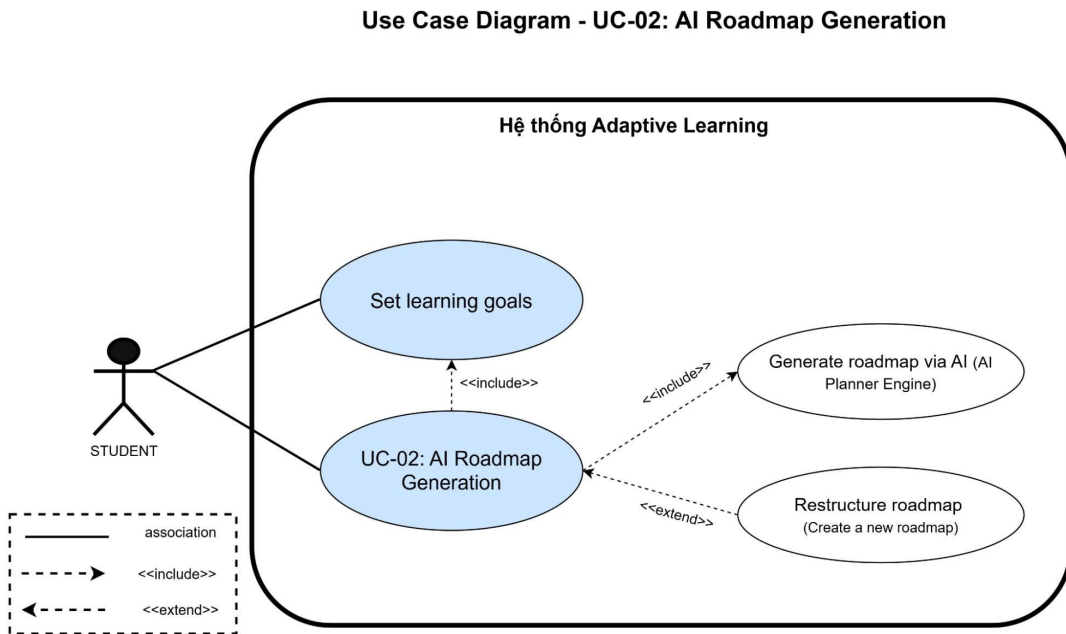


Figure 3: Use Case Diagram — UC-02: AI Roadmap Generation

### 3.2.3 Core Flow 3 (UC-03): Real-Time Adaptive Learning Loop

**Actors:** Student, Adaptive Engine (IRT Module), Question Bank DB, Learning Database.

### Pre-conditions:

- Student is authenticated via valid JWT Token.
- Student has an active Study Plan with generated Daily Tasks.
- Adaptive Engine (IRT Module), Question Bank DB, and Learning Database are online.

### Basic Flow:

1. Student starts a Daily Task session.

2. Adaptive Engine retrieves current topic ability ( $\theta$ ) and fetches first question matching difficulty  $b \approx \theta$ .
3. Student submits answer; system records choice and thinking time ( $T_{\text{think}}$ ).
4. Adaptive Engine evaluates correctness and updates temporary ability  $\theta_{\text{temp}}$  via Item Response Theory (IRT):
  - **Correct:**  $\theta_{\text{temp}}$  increases; next question selected has higher difficulty ( $b_{\text{next}} > b_{\text{current}}$ ).
  - **Incorrect:**  $\theta_{\text{temp}}$  decreases; next question selected has lower difficulty.
5. Loop repeats for 10–15 items; upon completion, final ability delta  $\Delta\theta$  updates Learning Profile officially.

**Alternative Flow:**

- **AF-3.1 Frustration Detection:** If student fails 3 consecutive items at minimum difficulty, system pauses test, offers theory summary, or redirects to AI Tutor (UC-04).

**Post-conditions:**

- **Success:** Question responses and thinking times are logged; Student Proficiency Level ( $\theta$ ) is updated in Learning Profile; task progress is marked in the Study Plan.
- **Failure:** Session state is saved; error logs are recorded upon engine failure.

### 3.2.4 Core Flow 4 (UC-04): AI Tutor Support with RAG

**Actors:** Student, AI Tutor Engine (LLM), RAG Engine, Vector Database, Teacher.

**Pre-conditions:**

- Student is authenticated via valid JWT Token.
- Knowledge base and theory documents are indexed in Vector Database.
- RAG Engine and LLM Service are online and reachable.

**Basic Flow:**

1. Student requests explanation for a failed exercise or enters a free-text prompt.
2. Prompt payload (question, chosen wrong answer, topic context) is vectorized into embeddings.
3. RAG Engine performs Cosine Similarity Search on Vector DB to retrieve Top-K ( $K = 3$ ) relevant theory chunks and re-ranks them.
4. Retrieved Context + System Prompt + Conversation History are sent to LLM Engine.

5. AI Tutor streams pedagogical response character-by-character via Server-Sent Events (SSE) with LaTeX and Markdown formatting.
6. System appends a “Similar Question Widget” at the end for immediate practice.

#### Alternative Flows:

- **AF-4.1 Low Similarity** ( $< 0.65$ ): To prevent AI Hallucination, system refrains from guessing, alerts user, and creates a support ticket for a human Teacher.
- **AF-4.2 Guardrail Trigger**: Out-of-scope prompts are gracefully rejected with a friendly domain restriction message.

#### Post-conditions:

- **Success**: Streaming pedagogical response with rendered math formulas is delivered to Student; chat interaction is logged into conversation history.
- **Failure**: Query is rejected or routed to human Teacher support ticket without generating hallucinated content.

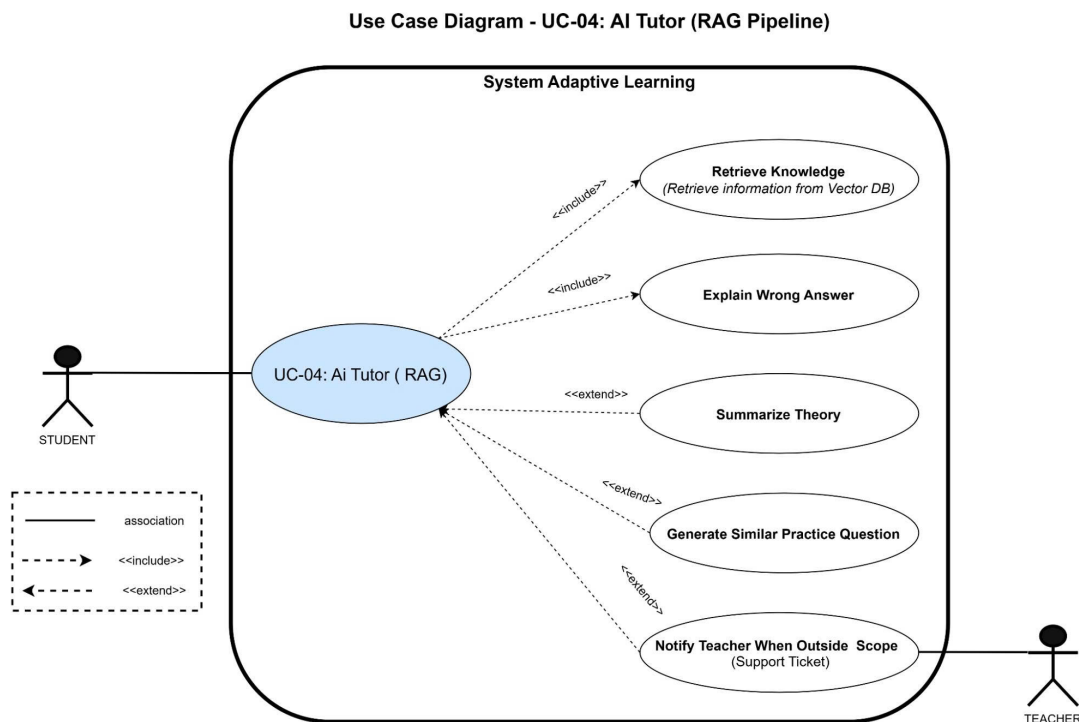


Figure 4: Use Case Diagram — UC-04: AI Tutor (RAG Pipeline)

### 3.2.5 Core Flow 5 (UC-05): Learning Analytics and Progress Monitoring

**Actors:** Student, Teacher, Parent, AI Analytics Engine, Learning Database.

#### Pre-conditions:

- User (Student, Teacher, or Parent) is authenticated via valid JWT Token.
- User role is verified via RBAC with appropriate access scope.
- Learning Database contains historical study records, quiz logs, and activity metrics.

**Basic Flow:**

1. Actor logs in and requests Learning Analytics Dashboard.
2. System checks JWT & RBAC scope (Student sees own data; Teacher sees assigned class; Parent sees linked child).
3. AI Analytics Engine aggregates metrics: Total Duration, Consistency, Quiz Accuracy, Competency Improvement, and Engagement.
4. Dashboard renders role-specific visual charts (Radar Charts, Progress Bars, Comparison Grids).
5. If metrics drop below warning thresholds, system automatically generates Learning Alerts.

**Post-conditions:**

- **Success:** Analytics dashboard renders accurate metrics, radar charts, and progress bars matching user role; learning alerts are dispatched if metrics breach thresholds.
- **Failure:** Access is denied if permissions mismatch; data loading error is displayed if database is unreachable.

### 3.2.6 Core Flow 6 (UC-06): Examination Performance Prediction

**Actors:** Student, Teacher, Parent, AI Prediction Engine, Learning Database.

**Pre-conditions:**

- User is authenticated and authorized under valid RBAC role.
- Sufficient historical assessment and practice data are recorded in Learning Database.
- AI Prediction Engine and LLM Service are online and reachable.

**Basic Flow:**

1. Triggered on-demand or automatically post-test.
2. AI Prediction Engine aggregates Knowledge Tracing records, Practice Exam scores, and Behavioral Consistency.
3. Combined models (Knowledge Tracing + Regression + Behavioral Analytics) predict final score and probability of achieving  $S_{\text{target}}$ .

4. LLM generates personalized natural language recommendations and risk assessments.
5. Dashboard displays predicted score ranges, weak topic highlights, and roadmap adjustment recommendations.

**Post-conditions:**

- **Success:** Predicted score range and probability of achieving Target Score ( $S_{\text{target}}$ ) are calculated and displayed on Dashboard along with AI recommendations.
- **Failure:** System displays an alert indicating insufficient historical data for prediction or records prediction service failure logs.

### 3.3 Non-Functional Requirements (NFR)

| ID     | Category    | Requirement Description                                                                                |
|--------|-------------|--------------------------------------------------------------------------------------------------------|
| NFR-01 | Performance | Average response time for standard operations (login, test load, dashboard) shall be $< 3$ seconds.    |
| NFR-02 | Performance | AI Tutor Time To First Token (TTFT) shall be $< 1.5$ seconds using SSE streaming.                      |
| NFR-03 | Performance | Ensure high data integrity and throughput without data loss during high-concurrency exam submissions.  |
| NFR-04 | Security    | User authentication must utilize JSON Web Tokens (JWT) with secure expiration.                         |
| NFR-05 | Security    | Role-Based Access Control (RBAC) must strictly segregate Admin, Teacher, Student, and Parent roles.    |
| NFR-06 | Security    | All data in transit must be encrypted using HTTPS/TLS protocols.                                       |
| NFR-07 | Security    | Input prompts must undergo Prompt Sanitization to prevent Prompt Injection attacks.                    |
| NFR-08 | Security    | Audit Logging must record all administrative actions and sensitive data access.                        |
| NFR-09 | Scalability | System must support at least 1,000 concurrent active users.                                            |
| NFR-10 | Scalability | Backend services must support Horizontal Scaling on Cloud infrastructure without service interruption. |

| ID     | Category        | Requirement Description                                                                                                |
|--------|-----------------|------------------------------------------------------------------------------------------------------------------------|
| NFR-11 | Architecture    | Cloud-native architecture deployed on Cloud platforms (e.g., Azure) with modular frontend/backend separation.          |
| NFR-12 | Integration     | Expose RESTful Open APIs for future Learning Management System (LMS) integrations.                                     |
| NFR-13 | Data Management | User data, learning profiles, and test logs must be securely backed up periodically with disaster recovery mechanisms. |

## 4 Software Design Description (SDD)

### 4.1 System Architecture Design

#### 4.1.1 High-Level System Architecture

The system is built upon a cloud-native, microservices-inspired architecture designed to deliver scalable, real-time adaptive learning and high-precision AI assistance. The architecture is logically divided into four main layers:

- **Presentation Layer (Frontend):** Developed using modern web and mobile frameworks (React.js / Flutter) serving customized interfaces for Students, Teachers, Parents, and System Administrators.
- **API Gateway & Authentication Layer:** Manages routing, rate limiting, and request validation using JSON Web Tokens (JWT) and Role-Based Access Control (RBAC).
- **Business & AI Core Logic Layer:**
  - *Adaptive Learning Engine:* Computes real-time student ability ( $\theta$ ) using Item Response Theory (IRT) algorithms.
  - *RAG & LLM Engine:* Connects with external Vector Databases via semantic Cosine Similarity search to stream verified pedagogical explanations through Server-Sent Events (SSE).
  - *Analytics & Prediction Service:* Aggregates Knowledge Tracing and behavioral metrics for score prediction and risk assessment.
- **Data Layer:** Combines relational database systems (PostgreSQL / MySQL for structured operational data) and specialized Vector Databases (Milvus / Pinecone for high-dimensional text embeddings).

### **4.1.2 Package Diagram**

The system codebase and architectural packages are decoupled into independent modules to ensure horizontal scalability and maintainability:

- `com.platform.auth`: Manages JWT lifecycle, RBAC enforcement, and audit logs.
- `com.platform.assessment`: Handles diagnostic tests, item scoring, and IRT ability calculations.
- `com.platform.ai.rag`: Orchestrates document vectorization, Cosine similarity search, and LLM prompt templates.
- `com.platform.analytics`: Processes learning logs, consistency metrics, and examination performance predictions.

## **4.2 Use Case Diagrams Design**

### **4.2.1 Overall System Use Case Diagram**

The high-level Use Case Diagram illustrates the functional boundary of the AI-Integrated Competency Assessment Prep System and the primary interactions among all four system actors: Student, Teacher, Parent, and System Administrator.

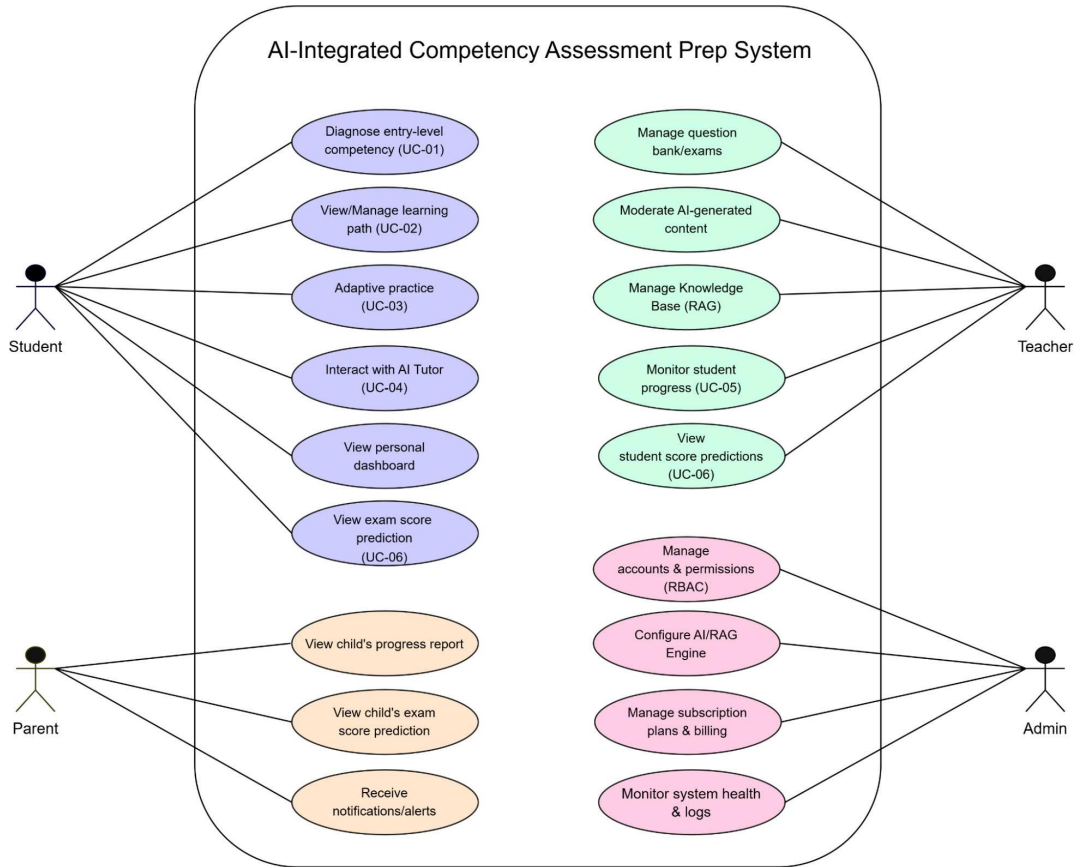


Figure 5: Overall System Use Case Diagram

#### 4.2.2 Student & AI Interactive Use Case Diagram

This diagram details the core interactive workflows for the Student actor, encompassing diagnostic assessment (UC-01), roadmap generation (UC-02), real-time adaptive practice loops (UC-03), RAG-powered AI Tutor interactions (UC-04), personal dashboards (UC-05), and predicted exam scores (UC-06).

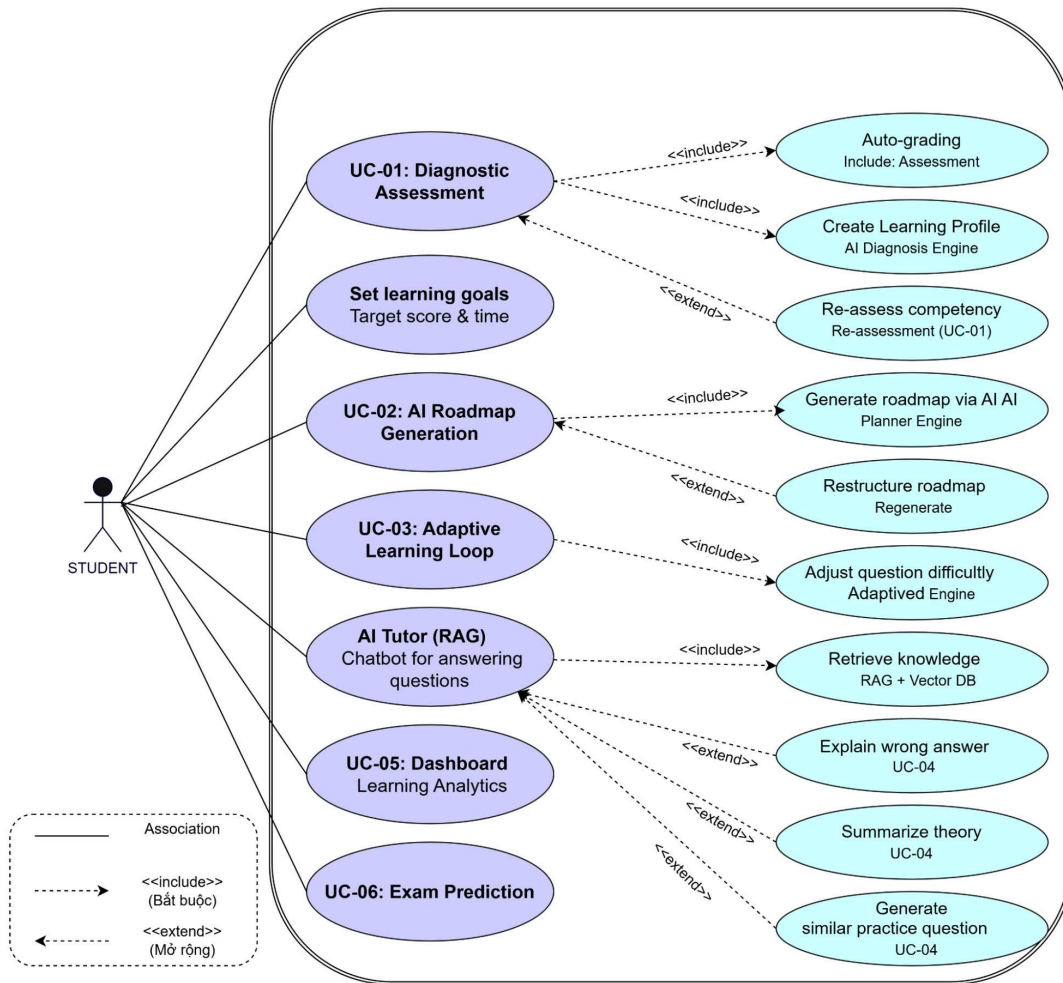


Figure 6: Student & AI Interactive Use Case Diagram

#### 4.2.3 Teacher & Admin Use Case Diagram

This detailed diagram depicts administrative and academic management functions. System Administrators configure AI parameters, RBAC roles, and system security. Teachers manage standardized question banks, upload educational documents to the Vector Database, and moderate AI-generated responses (Human-in-the-loop).



Figure 7: Teacher & Admin Use Case Diagram

#### 4.2.4 Parent Use Case Diagram

This diagram outlines the read-only monitoring functionalities for Parent accounts, including progress tracking, attendance/inactivity alerts, and automated performance reports.

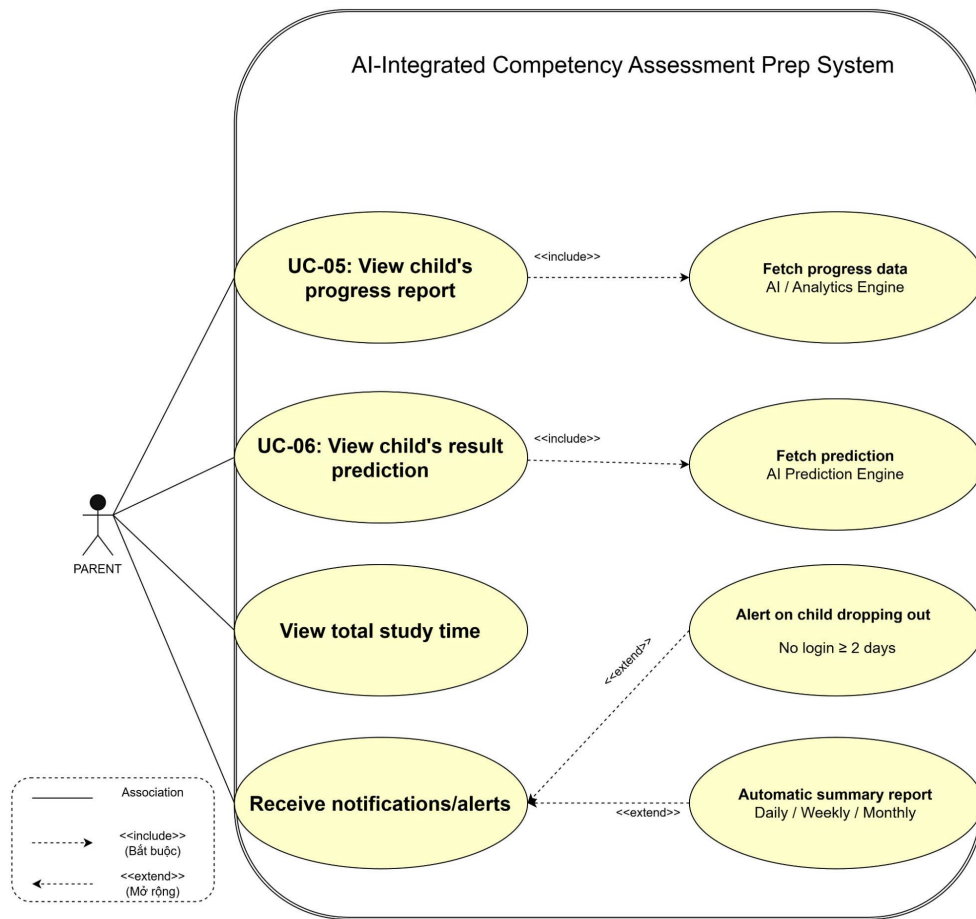


Figure 8: Parent Use Case Diagram

## 4.3 Database Design (ERD & Vector DB)

### 4.3.1 Relational Database Schema (ERD)

The relational database manages operational data including Users, Roles, Question Banks, Diagnostic Sessions, Learning Profiles, and Roadmap Daily Tasks. Strict foreign key constraints and index optimizations ensure data integrity and fast response times ( $< 3.0s$ ).

### 4.3.2 Vector Database Architecture & AI RAG Pipeline

The system utilizes an in-memory / relational vector store for baseline semantic retrieval. Text-books and solution documents are processed into chunks and stored in the `vector_chunks` table:

- `chunk_id` (UUID, Primary Key): Unique identifier for each text chunk.
- `document_id` (String, Foreign Key): Identifier of the original source document.
- `text_content` (Text): Raw textual context extracted from documents.
- `embedding_vector` (Array[Float]): Dense L2-normalized vector array.

- `chunk_index` (Integer): Sequence position of the chunk in the document.

#### 4.3.3 AI RAG Processing Algorithms

- **Text Chunker (`text_chunker`):** Splits source documents using a sliding window algorithm with a default `chunk_size` of 100 words and an overlap of 20 words.
- **Vector Embedding & L2 Normalization (`generate_embedding`):** Converts plain text into numeric vectors and scales them to unit length ( $\|v\| = 1.0$ ) for optimized vector operations.
- **Cosine Similarity & Top-K Retrieval (`retrieve_relevant_chunks`):** Computes angular similarity between user query vectors and chunk vectors to retrieve the Top-K ( $K = 3$ ) most relevant contexts for the AI Tutor.

### 4.4 Sequence Diagrams & Detailed Design

Sequence diagrams describe the chronological object interactions across the Frontend, Backend API Gateway, IRT Engine, RAG Service, Vector DB, and LLM API for critical business flows:

- **SD-01:** Real-time Item Response Adjustment during Adaptive Practice (UC-03).
- **SD-02:** RAG Context Retrieval and Streaming Response Generation for AI Tutor Chatbot (UC-04).

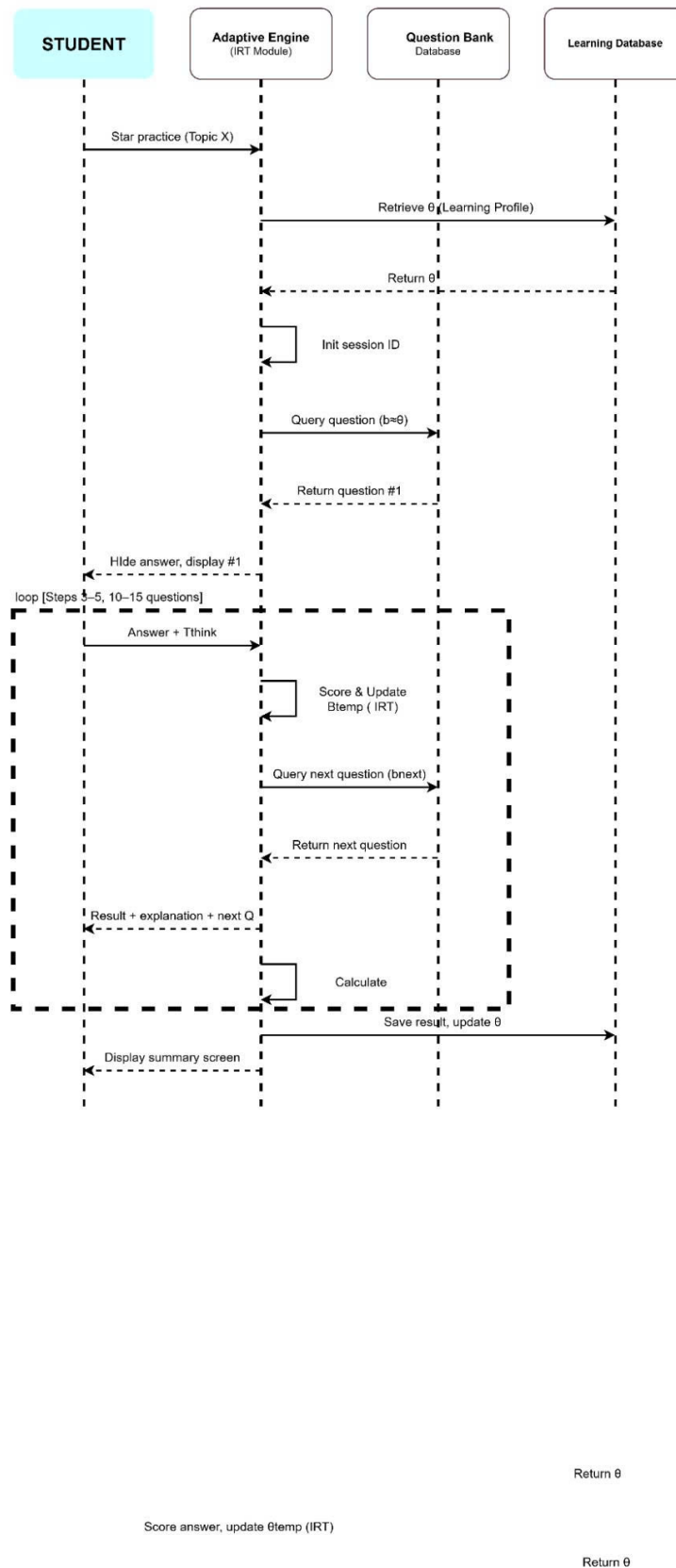


Figure 9: SD-01: Real-time Item Response Adjustment during Adaptive Practice (UC-03)

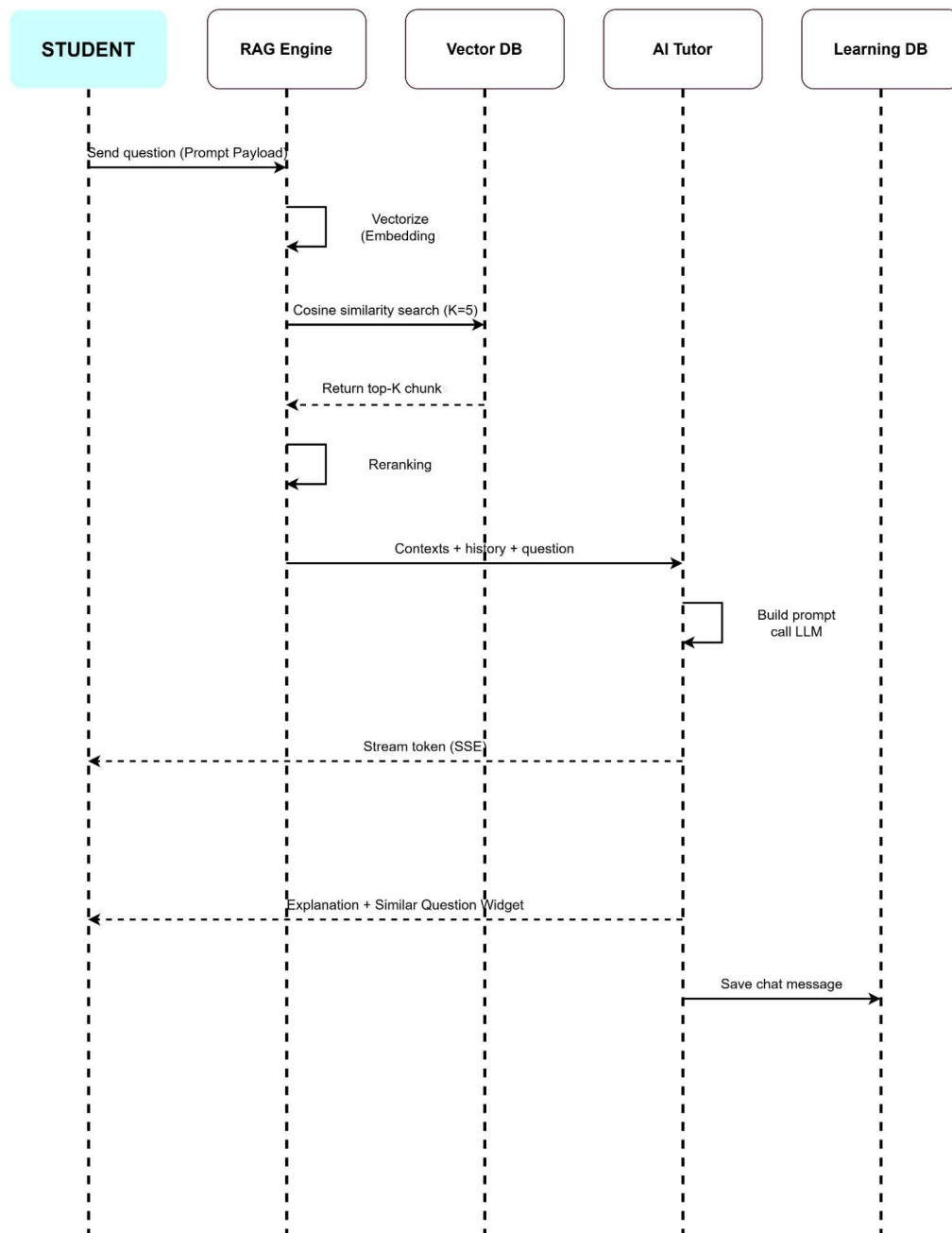


Figure 10: SD-02: RAG Context Retrieval and Streaming Response Generation for AI Tutor Chatbot (UC-04)

## 5 System Implementation & Testing

### 5.1 Implementation Tech Stack

- **Backend Core:** Python 3.10+, FastAPI framework, SQLAlchemy ORM (async/sync sessions), Pydantic v2 data validation schemas.
- **Security & Authentication:** JWT Bearer Token Authentication (HS256), Bcrypt crypto-

graphic password hashing, Role-Based Access Control (RBAC) dependency injection middleware.

- **Database & Storage:** PostgreSQL / Supabase for relational operational persistence; in-memory vector embeddings with Cosine Similarity for RAG retrieval.
- **Artificial Intelligence Engine:** Google Gemini API (gemini-1.5-flash) integrated via asynchronous streaming with Socratic pedagogical system prompts.
- **Frontend Application:** React.js 19, Vite build tool, Material-UI (MUI v7), React Router v7, KaTeX math typesetting engine (rehype-katex, remark-math).
- **Automated Testing Suite:** Pytest testing framework with isolated SQLite test database fixtures and unittest.mock asynchronous stubs.

## 5.2 System Features & User Interface Walkthrough

### 5.2.1 Authentication & Multi-Role Access Control (RBAC)

- **Functional Description:** Serves as the primary security gateway for the platform. It enforces distinct operational boundaries across the four primary system personas: Student, Teacher, Parent, and System Administrator.
- **User Workflow & Validation Rules:**
  1. Users access the unified authentication screen at /login.
  2. The form validates email formatting via RFC-compliant regex and mandates a minimum password length of 8 characters.
  3. Upon successful credential verification, the backend issues an access\_token (30-minute lifespan) and a refresh\_token (7-day lifespan).
  4. The frontend context automatically decodes user role claims and routes the client to the appropriate workspace (/student, /teacher, /parent, or /admin).
- **Underlying APIs:** POST /api/v1/auth/register, POST /api/v1/auth/login, GET /api/v1/auth/me.

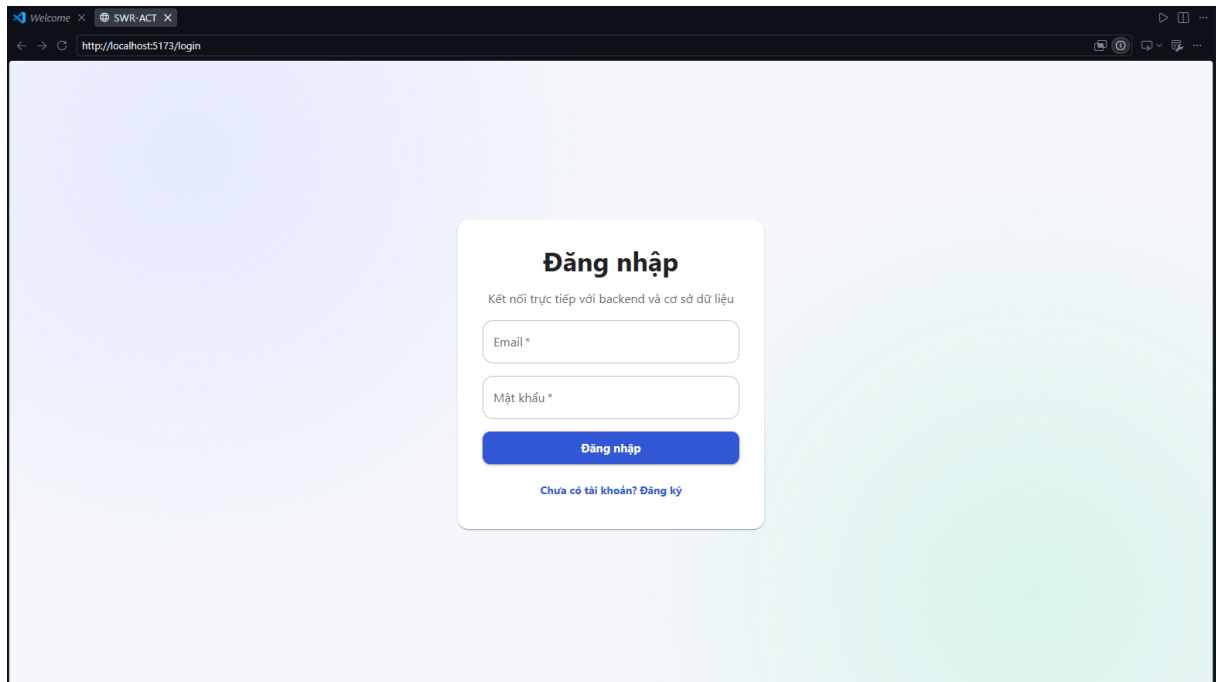


Figure 11: User Authentication and Role-Based Redirection Interface

### 5.2.2 Competency Diagnosis & Baseline Assessment Module (UC-01)

- **Functional Description:** Evaluates the student's baseline academic capability across multiple knowledge domains (Mathematics, Logic Reasoning, Vietnamese Language) to identify existing knowledge gaps before generating a study plan.
- **User Workflow & UI Components:**
  1. The student initiates a diagnostic session via `/student/assessment`.
  2. The system loads 15 standardized competency evaluation items into an interactive examination room.
  3. A real-time countdown timer (15:00 minutes) and linear progress bar dynamically track examination progress.
  4. Mathematical expressions and equations are rendered natively using KaTeX.
  5. Upon submission (manual or auto-submitted upon timer expiration), the system calculates raw score metrics and presents a granular review showing selected options, correct keys, and comprehensive explanations.
- **Underlying APIs:** GET `/api/diagnostic/questions`, POST `/api/diagnostic/submit`.

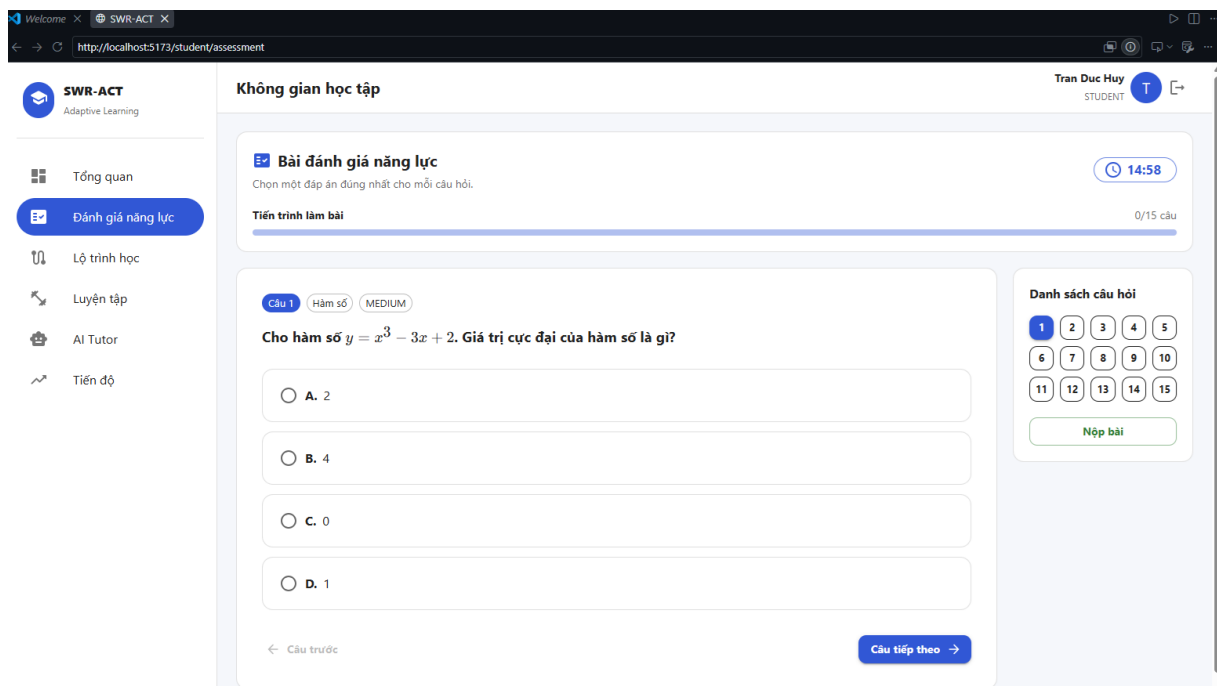


Figure 12: Diagnostic Assessment Exam Room with KaTeX Math Rendering and Countdown Timer

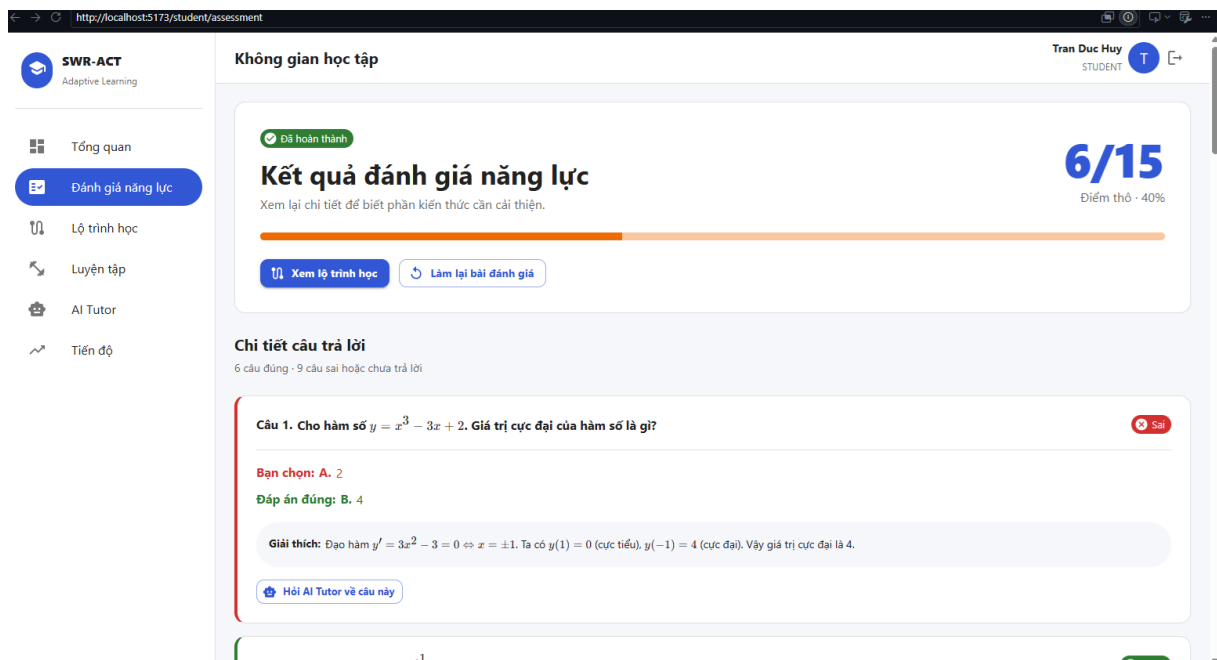


Figure 13: Diagnostic Evaluation Summary and Detailed Explanations View

### 5.2.3 Goal Setting & Personalized Learning Roadmap Module (UC-02)

- **Functional Description:** Converts diagnostic test outcomes into actionable, daily study milestones customized to the student's target competency score and preparation timeframe.
- **User Workflow & UI Components:**
  1. The student accesses the roadmap screen at /student/roadmap and opens the Goal Setup Modal.
  2. Input constraints are validated in real-time:
    - Target Score ( $S_{\text{target}}$ ): Range 1 to 1200 points.
    - Exam Date ( $D_{\text{exam}}$ ): Must be a valid future date.
    - Daily Study Time ( $T_{\text{daily}}$ ): Range 0.5 to 12.0 hours/day.
  3. The backend calculates total remaining preparation days and initializes a structured 4-phase learning trajectory stored in study\_plans and plan\_tasks.
- **Underlying APIs:** POST /api/study-plan/init.

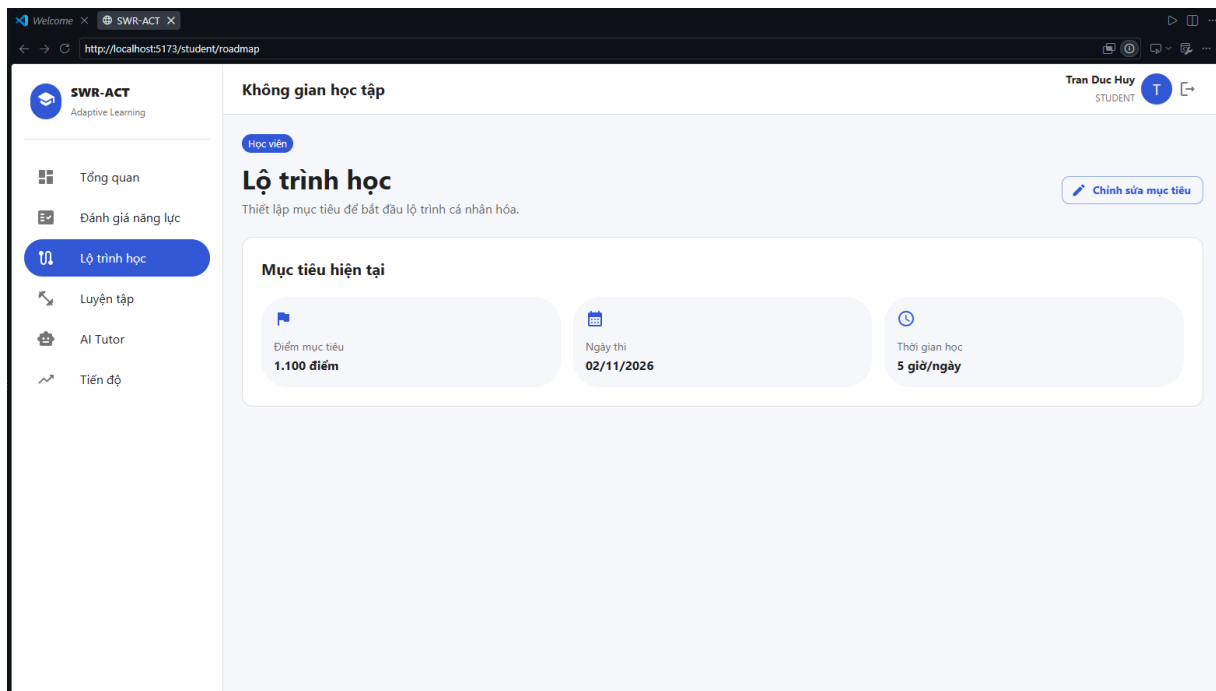


Figure 14: Goal Setup Modal and Active Milestone Trajectory Dashboard

### 5.2.4 Topic-Based Adaptive Practice Module (UC-03)

- **Functional Description:** Allows students to engage in targeted practice sessions filtered by specific subject areas, topic clusters, and difficulty tiers to reinforce weak competencies.

- **User Workflow & UI Components:**

1. The student configures a session on the Practice page by selecting:
  - Subject: Toan hoc , Logic , Tiếng Viet
  - Topic: Ham so , tinh phan , menh de , ...
  - Difficulty: EASY, MEDIUM, HARD.
2. The system retrieves matched question items from the database.
3. The exam room provides a numeric question navigator grid, allowing students to track answered/unanswered questions seamlessly.
4. Post-submission results persist into tests and user\_answers, generating a performance breakdown with a dedicated “Hoi AI cau nay” (Ask AI Tutor) button next to every incorrect answer.

- **Underlying APIs:** GET /api/practice/questions, POST /api/practice/submit, GET /api/practice/history.

The screenshot displays the 'Cấu hình bài luyện tập' (Configure Practice Session) interface. It features a sidebar with navigation links: Tổng quan, Đánh giá năng lực, Lộ trình học, Luyện tập (highlighted), AI Tutor, and Tiến độ. The main content area is titled 'Không gian học tập' and 'Học viên'. It includes three configuration steps: 1. Chọn môn thi (Choose exam subject) with buttons for Toán (Đại số, giải tích và xác suất), Logic (Quy luật và tư duy suy luận), and Tiếng Việt (Ngôn ngữ và đọc hiểu văn bản); 2. Chọn chuyên đề (Choose topic) with a dropdown menu set to 'Tất cả chuyên đề'; 3. Chọn mức độ khó (Choose difficulty level) with radio buttons for Dễ (Củng cố kiến thức nền tảng), Trung bình (Luyện tập ở mức tiêu chuẩn - selected), and Khó (Thử thách khả năng vận dụng). At the bottom right, there are buttons for 'Đặt lại' and 'Tiếp tục'.

Figure 15: Topic and Difficulty Configuration for Adaptive Practice Sessions

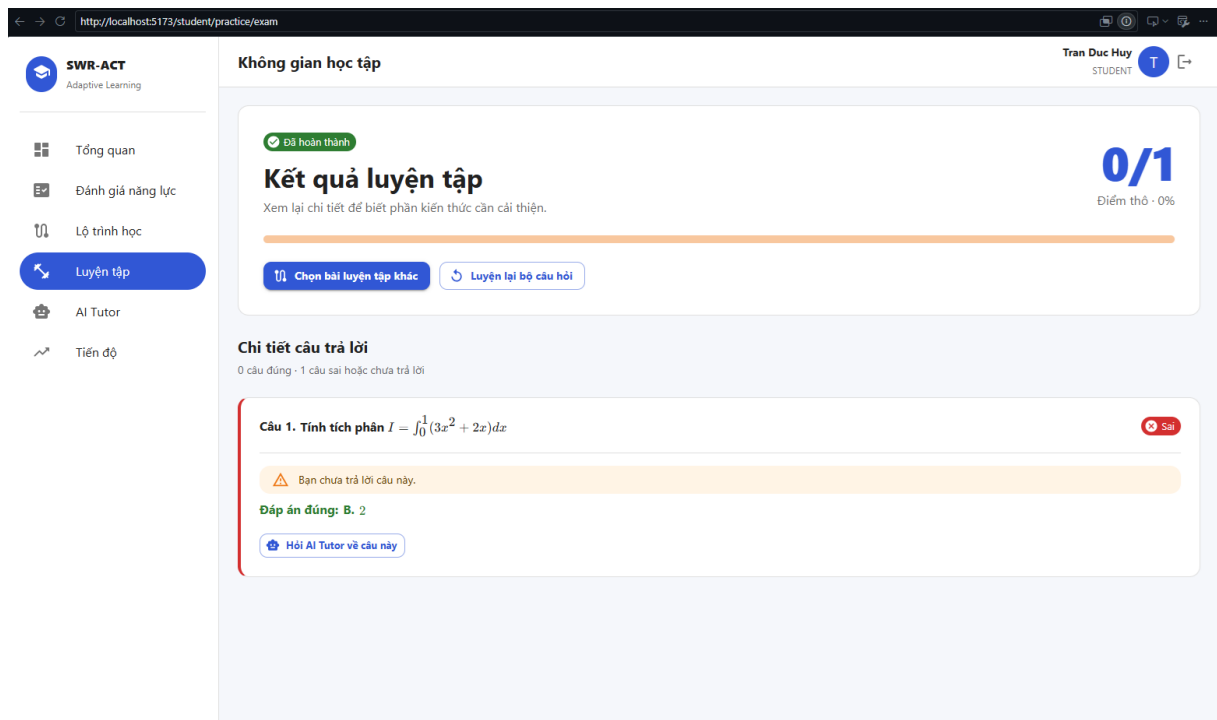


Figure 16: Practice Result Breakdown with 'Ask AI Tutor' Integration Button

### 5.2.5 Intelligent AI Tutor with RAG Integration (UC-04)

- **Functional Description:** Provides 24/7 Socratic pedagogical guidance. Instead of giving direct answers, the AI Tutor analyzes the student's selected wrong answer against the official explanation to guide their critical thinking process.
- **User Workflow & UI Components:**
  1. Clicking "Hỏi AI câu này" on an incorrect question seamlessly redirects the student to /student/tutor while automatically carrying the question\_id and problem metadata.
  2. The backend constructs a structured prompt combining the question statement, choices, correct key, official solution, and system pedagogical guardrails.
  3. The AI generates step-by-step hints and conceptual explanations formatted in Markdown and LaTeX.
- **Underlying APIs:** POST /api/v1/rbac/tutor/ask.

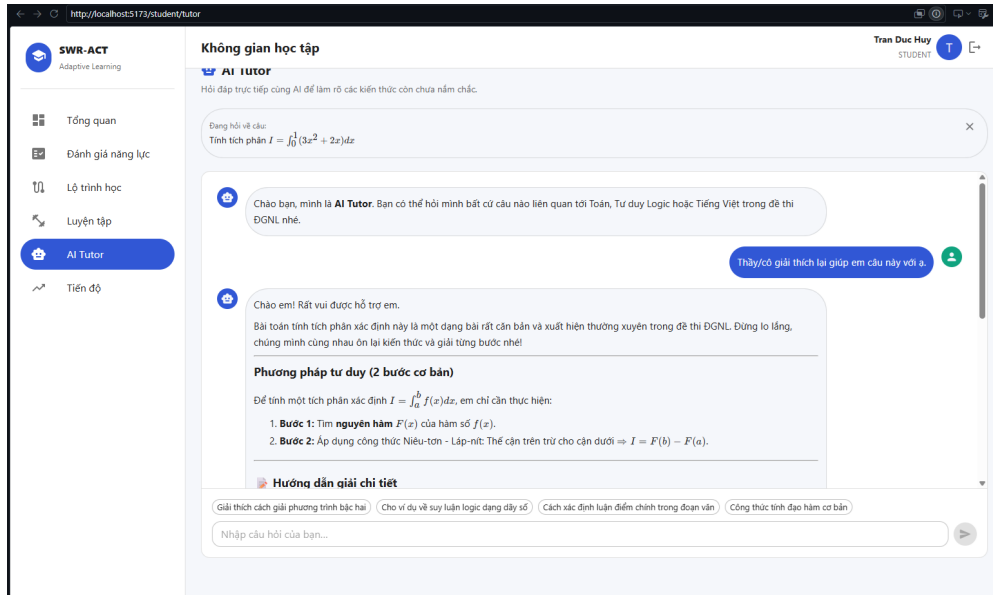


Figure 17: AI Tutor Pedagogical Explanation Interface with KaTeX Formulas

### 5.2.6 Student Overview & Competency Progress Analytics Module (UC-05)

- Functional Description:** Aggregates individual learning milestones and historical examination metrics into a unified progress dashboard. It delivers multidimensional competency profiling via radar charts and historical score progression curves to evaluate knowledge retention over time.
- User Workflow & UI Components:**
  - Overview Dashboard (/student):** Displays real-time countdown widgets to the examination date, target score card ( $S_{\text{target}}$ ), active daily study tasks retrieved via PlanTask, and recent practice test history logs.
  - Competency Radar Chart (/student/progress):** Utilizes Recharts to render an interactive Polar Radar Chart mapping student accuracy across three core examination pillars: Mathematics, Logic Reasoning, and Vietnamese Language.
  - Historical Score Trendline:** Renders a responsive Line Chart tracking daily test score percentages, paired with an algorithmic score prediction card ( $S_{\text{predict}} \in [1, 1200]$ ) derived from the mean and observed standard deviation of completed tests.
- Underlying APIs:** GET /api/study-plan/current, GET /api/practice/history, GET /api/analytics/progress

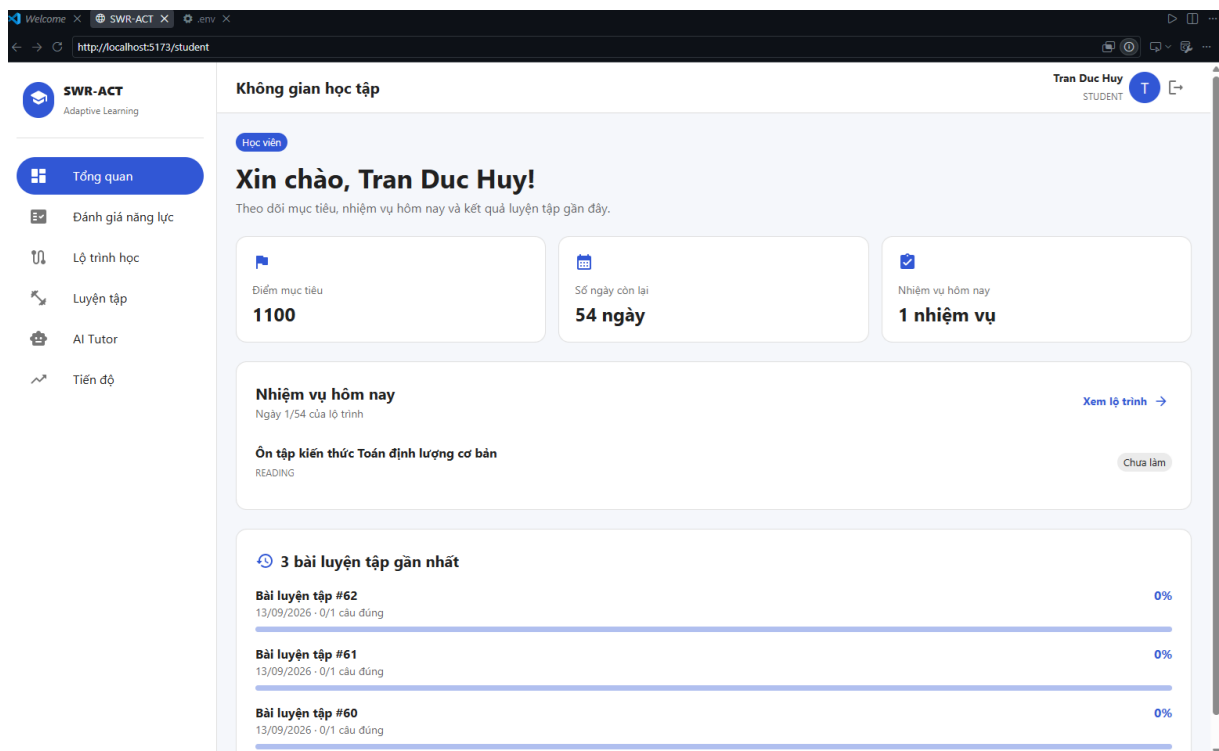


Figure 18: Student Overview Dashboard and Daily Learning Tasks Tracking

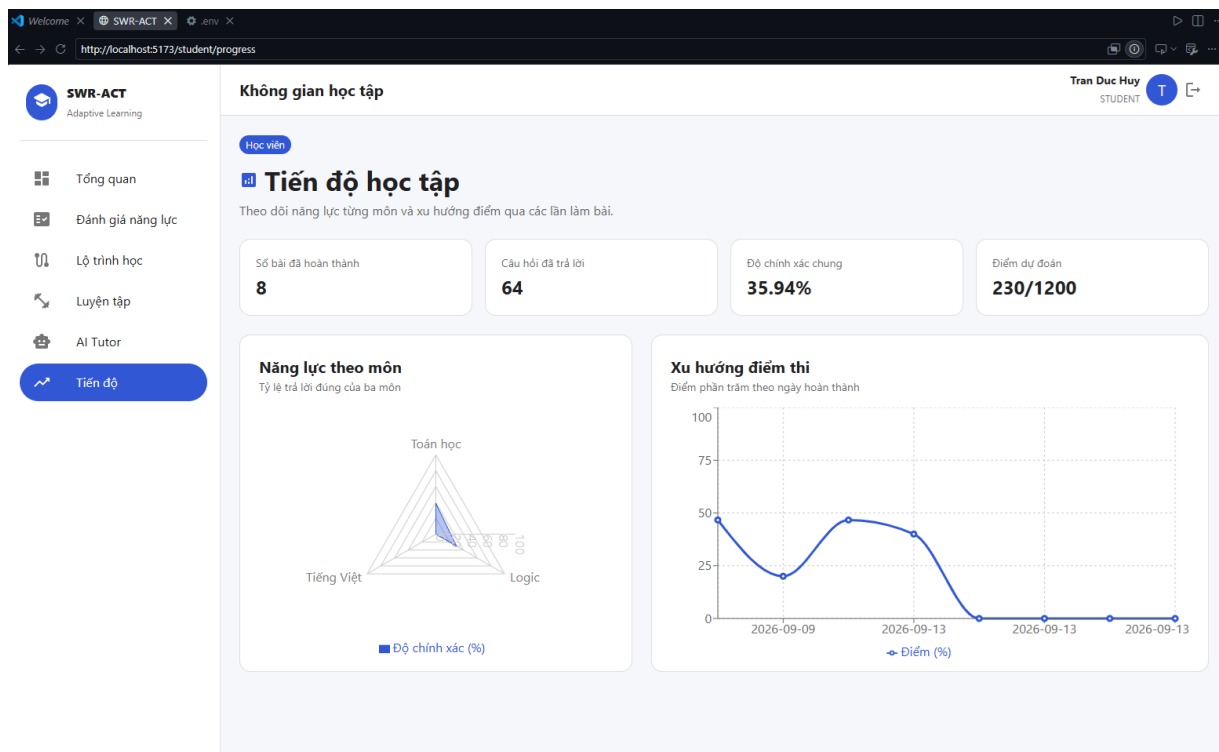


Figure 19: Student Competency Analytics with Radar Accuracy and Historical Score Curves

### 5.2.7 Teacher Portal: Question Bank Management System (CRUD) (UC-06)

- **Functional Description:** Grants pedagogical staff full administrative governance over the central question repository, enforcing multi-criteria search, pagination, and real-time mathematical validation before persistence.
- **User Workflow & UI Components:**
  - *Central Data Table (/teacher/questions):* Renders a responsive question bank table protected by TEACHER and ADMIN RBAC guards. Supports dynamic keyword search on question stems, subject filters, topic selectors, and difficulty chips (EASY, MEDIUM, HARD).
  - *Interactive Question Modal Form:* Empowers instructors to execute Create, Read, Update, and Delete (CRUD) operations. The form provides decoupled inputs for four choices (A, B, C, D), correct key designation, and detailed pedagogical rationale.
  - *Real-Time KaTeX Math Preview:* Features an integrated live preview toggle powered by MathMarkdown, ensuring complex mathematical equations, matrices, and LaTeX notation render without formatting errors prior to database submission.
- **Underlying APIs:** GET /api/teacher/questions, POST /api/teacher/questions, PUT /api/teacher/question, DELETE /api/teacher/questions/{id}.

The screenshot displays the 'Ngân hàng câu hỏi' (Question Bank) interface. It features a sidebar with navigation links: 'Tổng quan', 'Ngân hàng câu hỏi' (selected), 'Kho tri thức', and 'Phân tích học sinh'. The main content area has a header with the title 'Ngân hàng câu hỏi' and a subtitle 'Quản lý câu hỏi trắc nghiệm: thêm, sửa, xóa và lọc theo môn học'. Below the header is a search bar and filter controls for 'Môn học', 'Chủ đề', and 'Độ khó'. The table below lists questions with columns: 'Nội dung', 'Môn học', 'Chủ đề', 'Độ khó', 'Đáp án', and 'Thao tác'.

| Nội dung                                                                 | Môn học  | Chủ đề          | Độ khó     | Đáp án | Thao tác |
|--------------------------------------------------------------------------|----------|-----------------|------------|--------|----------|
| Cho hàm số $y = x^3 - 3x + 2$ . Giá trị cực đại của hàm số là gì?        | Toán học | Hàm số          | Trung bình | B      |          |
| Tính tích phân $\int_0^1 (2x + 1) dx$ .                                  | Toán học | Tích phân       | Dễ         | B      |          |
| Cho cấp số cộng có $u_1 = 2$ và công sai $d = 3$ . Tìm số hạng $...$     | Toán học | Dãy số          | Dễ         | A      |          |
| Có bao nhiêu cách xếp 5 học sinh thành một hàng ngang?                   | Toán học | Tổ hợp          | Trung bình | B      |          |
| Xác suất để gieo một con súc sắc cân đối xuất hiện mặt chấm chẵn...      | Toán học | Xác suất        | Dễ         | A      |          |
| Tìm giá trị lớn nhất của hàm số $y = -x^2 + 4x + 1$ trên đoạn $[0, ...]$ | Toán học | Hàm số          | Dễ         | B      |          |
| Một lớp có 20 học sinh nam và 15 học sinh nữ. Chọn ngẫu nhiên ...        | Toán học | Xác suất        | Trung bình | C      |          |
| Cho hình lăng trụ tam giác đều $ABC.A'B'C'$ có tất cả các cạnh b...      | Toán học | Hình không gian | Trung bình | A      |          |
| Tọa độ của bất phương trình $2(x - 1) < 3$ là gì?                        | Toán học | Mô - Logarit    | Dễ         | B      |          |

Figure 20: Teacher Question Bank Data Table with Filter and Pagination Controls

Không gian học tập

Thêm câu hỏi mới

Xem trước

Nội dung câu hỏi \*

Tính tích phân  $I = \int_0^1 (3x^2 + 2x) dx$ .

Hỗ trợ công thức LaTeX, ví dụ:  $5x^2 + 1 = 0$

Đáp án A \*

$1$

Đáp án B \*

$2$

Đáp án C \*

$\frac{5}{2}$

Đáp án D \*

$\frac{3}{2}$

Đáp án đúng \*

B

Môn học \*

Toán học

Chủ đề \*

Tích phân

Độ khó \*

Trung bình

Lời giải (không bắt buộc)

Hủy

Lưu câu hỏi

Figure 21: Question Modal Interface featuring Real-Time KaTeX Math Rendering Preview

?

Thêm câu hỏi mới

Xem trước

i

Xem trước công thức toán (LaTeX) hiển thị đúng như học sinh sẽ thấy.

Nội dung câu hỏi

Tính tích phân  $I = \int_0^1 (3x^2 + 2x)dx$ .

Đáp án A

1

Đáp án B

2

Đáp án C

$\frac{5}{2}$

Đáp án D

$\frac{3}{2}$

Hủy

Lưu câu hỏi

36

Một lớp có 20 học sinh nam và 15 học sinh nữ. Chọn ngẫu nhiên ...

Toán học

Xác suất

Trung bình

5.2.8 Teacher Portal: Class-Wide Performance Analytics (UC-07)

- **Functional Description:** Provides high-level cohort telemetry for instructors, summarizing student engagement, test completion volumes, and subject-level academic proficiencies across the entire student population.
- **User Workflow & UI Components:**
  - *Cohort Metric Cards (/teacher/analytics):* Summarizes primary system KPIs including Total Enrolled Students (role = STUDENT) and Total Completed Submissions (status = COMPLETED).
  - *Subject-Level Proficiency Distribution:* Visualizes normalized mean scores across knowledge domains, highlighting general performance baselines to assist teachers in adjusting classroom curriculum.
- **Underlying APIs:** GET /api/teacher/analytics/overview.

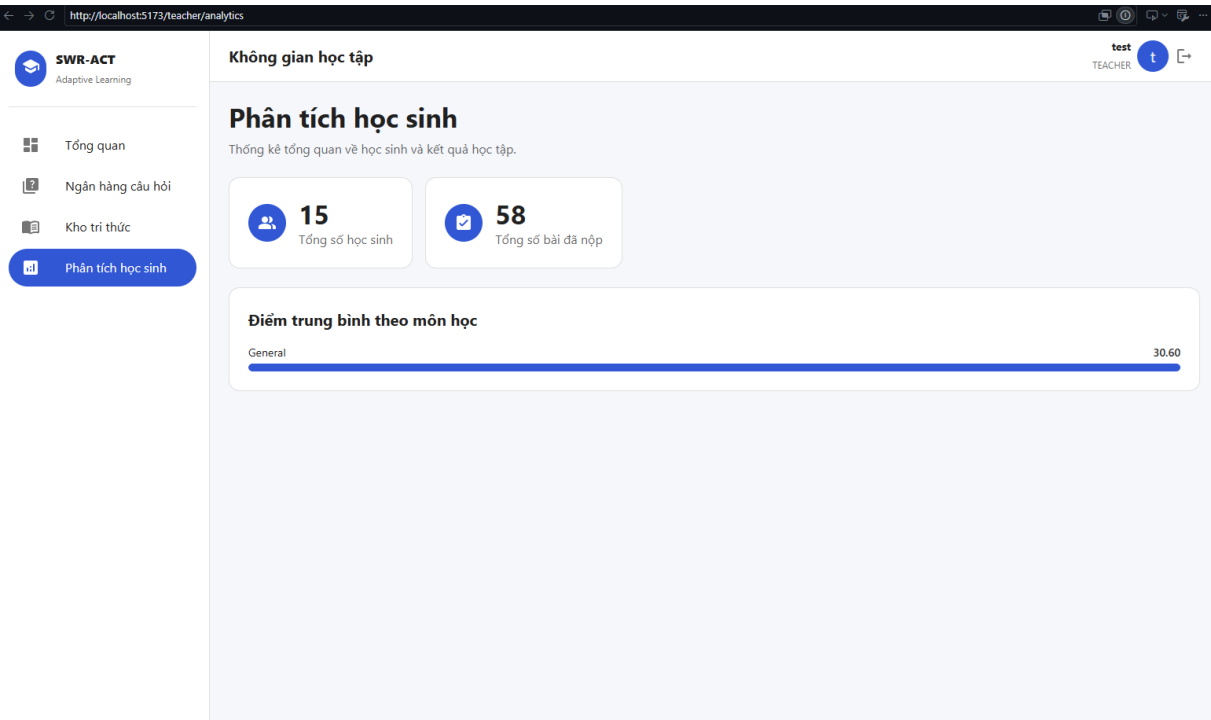


Figure 22: Teacher Class Performance Overview and Subject Score Distribution

5.2.9 Parent Portal Subsystem

- **Functional Description & System Architecture:** The Parent Portal module is designed to provide guardians with direct, transparent visibility into their children’s academic trajectory

and competency preparation for the National High School Graduation and Competency Assessment exams (DGNL). Secured under strict Role-Based Access Control (UserRole.PARENT), this subsystem queries relational mappings in the `student_parents` table and performs analytical aggregations against `study_plans`, `tests`, and `user_answers`.

- **Key Features Include:**

- *Academic Goal Tracking & Countdown:* Dynamically extracts the active study plan target score ( $S_{\text{target}} \in [1, 1200]$ ) and calculates remaining preparation days until target completion.
- *Multi-Subject Competency Radar:* Renders a balanced radar chart illustrating student proficiency across core domains: Mathematics, Logical Thinking, and Vietnamese Language.
- *Performance Trajectory & Prediction:* Visualizes chronological exam scores accompanied by dynamic upper/lower bound confidence intervals for real exam performance forecasting.

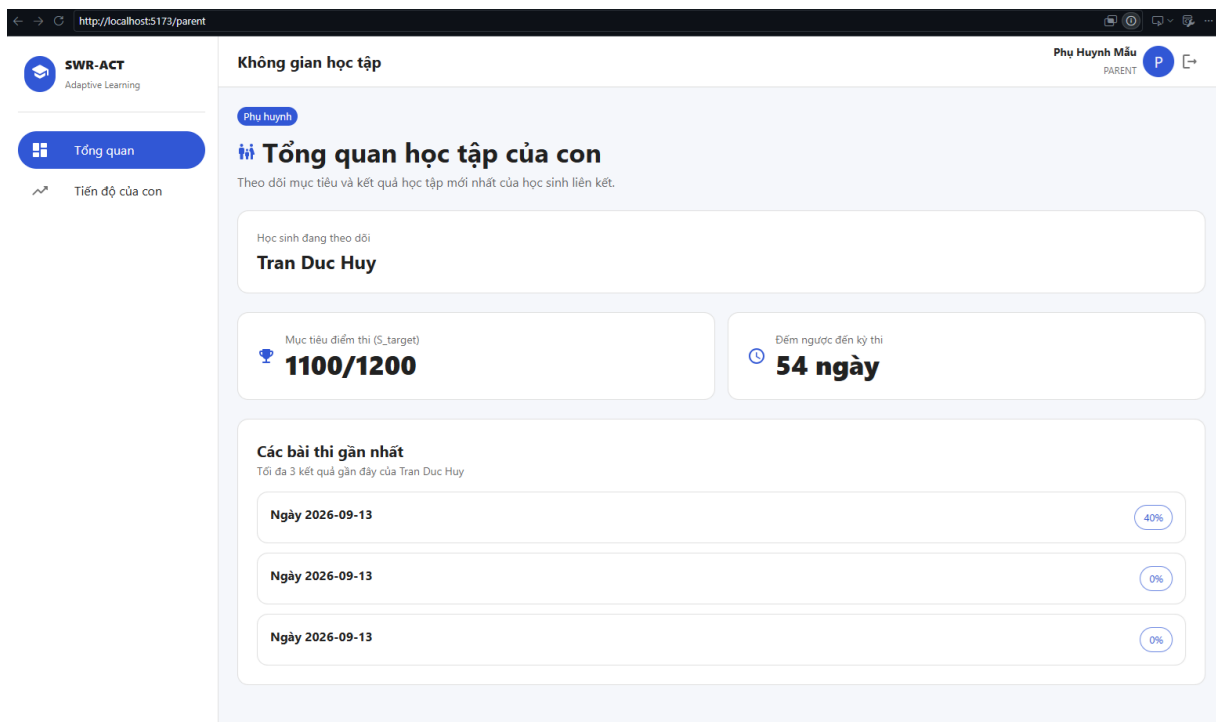


Figure 23: Parent Portal Overview Dashboard

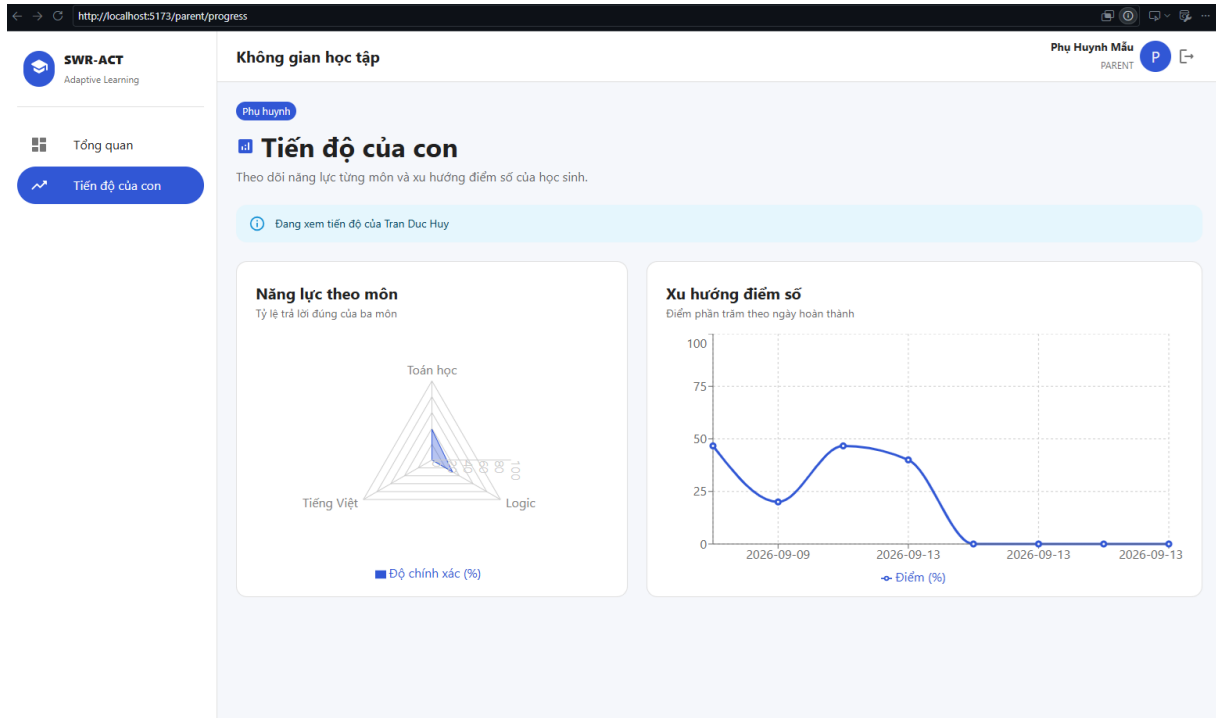


Figure 24: Parent Portal Progress Tracking with Multi-Subject Visualizations

### 5.2.10 Teacher Overview & Knowledge Base (RAG)

- **Functional Description & RAG Retrieval Pipeline:** The Teacher Management Subsystem equips instructional staff with macroscopic telemetry and semantic knowledge retrieval verification tools. Protected under `require_roles(UserRole.TEACHER, UserRole.ADMIN)`, this module couples aggregate administrative data with the core Retrieval-Augmented Generation (RAG) semantic pipeline.
- **Key Features Include:**
  - *Systemic Metrics Telemetry:* Delivers dynamic counter statistics on total question bank inventory, active student enrollment, and cumulative test submissions categorized by subject area.
  - *Semantic Knowledge Base (RAG Verification):* Connects to the `/api/v1/rag/query` pipeline. Educators provide reference pedagogical texts and query prompts; the backend executes sliding-window chunking, generates normalized embeddings, and returns top-ranked relevant passages scored by Cosine Similarity ( $S_c \in [-1.0, 1.0]$ ).
  - *Interoperability & Shared Engine:* Maintains backward-compatible authorization policies ensuring both educators (TEACHER) and learners (STUDENT) leverage the identical inference engine without RBAC collisions.

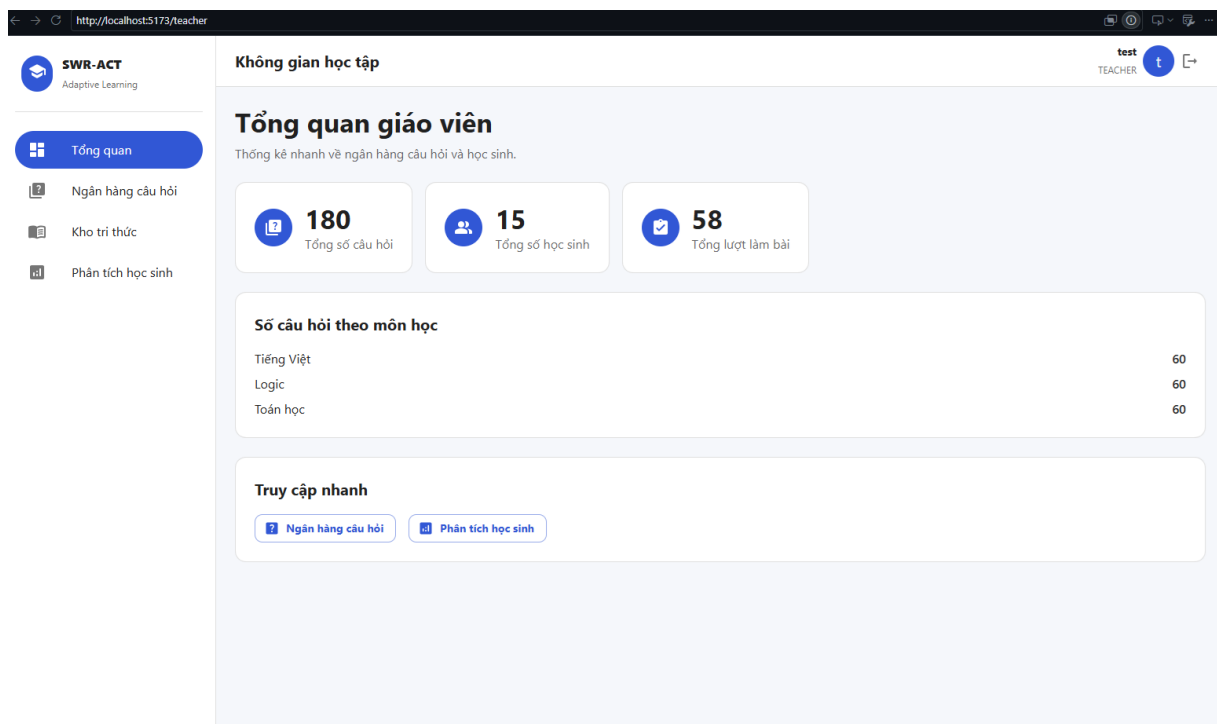


Figure 25: Teacher Overview Analytics Dashboard

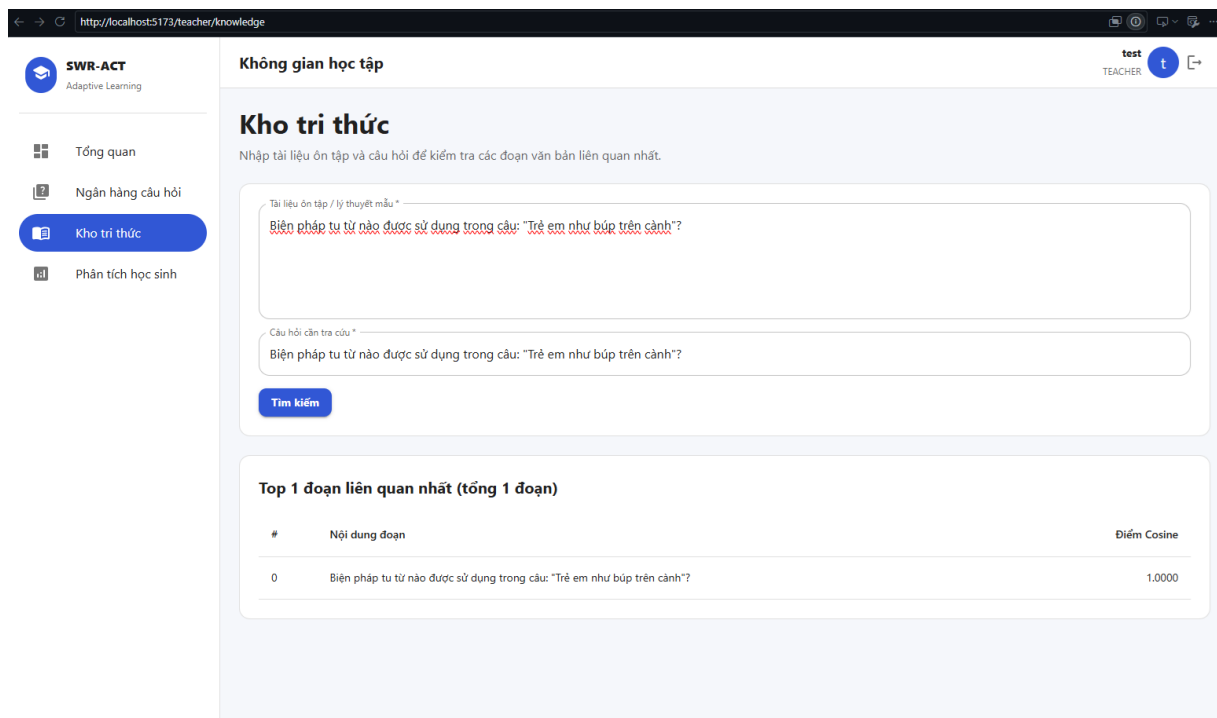


Figure 26: Teacher Knowledge Base Semantic Retrieval Verification Interface

## 5.3 Automated Unit & Integration Testing Results

To verify overall system stability, regression resilience, and ensure strict compliance across all core modules (Authentication, 4-Role RBAC, RAG Pipeline, Adaptive Practice, Diagnostic Assessment, Learning Roadmaps, Analytics Engine, Teacher Management, and Parent Portal Monitoring), an automated test suite was constructed and executed via Pytest.

- **Test Execution Result:** 56/56 Tests Passed (100% Pass Rate).
- **Total Execution Time:** ~8.06s seconds.
- **Test Environment:** Isolated SQLite Database engine with transactional session rollback fixtures and Asynchronous Mocking Stubs (unittest.mock.AsyncMock).

```
(.venv) PS C:\Users\Duc Hai\Desktop\Fix\Whom-5-CNPM\backend> python -m pytest -v
tests/test_study_plan_target_score.py::test_study_plan_target_score_rejects_out_of_range_values[0] PASSED [ 73%]
tests/test_study_plan_target_score.py::test_study_plan_target_score_rejects_out_of_range_values[1200.01] PASSED [ 75%]
tests/test_teacher_and_rag.py::test_teacher_overview_rbac_denied_for_student PASSED [ 76%]
tests/test_teacher_and_rag.py::test_teacher_overview_success PASSED [ 78%]
tests/test_teacher_and_rag.py::test_rag_query_rbac_allowed_for_student PASSED [ 80%]
tests/test_teacher_and_rag.py::test_rag_query_validation_empty_document PASSED [ 82%]
tests/test_teacher_and_rag.py::test_rag_query_validation_empty_query PASSED [ 83%]
tests/test_teacher_and_rag.py::test_rag_query_success PASSED [ 85%]
tests/test_teacher_api.py::test_teacher_endpoints_forbidden_for_student PASSED [ 87%]
tests/test_teacher_api.py::test_get_teacher_questions_with_filters PASSED [ 89%]
tests/test_teacher_api.py::test_create_update_delete_question_crud_flow PASSED [ 91%]
tests/test_teacher_api.py::test_get_teacher_analytics_overview PASSED [ 92%]
tests/test_tutor_api.py::test_ask_ai_tutor_unauthorized PASSED [ 94%]
tests/test_tutor_api.py::test_ask_ai_tutor_forbidden_role PASSED [ 96%]
tests/test_tutor_api.py::test_ask_ai_tutor_with_question_context_success PASSED [ 98%]
tests/test_tutor_api.py::test_ask_ai_tutor_question_not_found PASSED [100%]

===== warnings summary =====
..\venv\Lib\site-packages\fastapi\testclient.py:1
C:\Users\Duc Hai\Desktop\Fix\Whom-5-CNPM\venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa
app\rag\tutor_service.py:3
C:\Users\Duc Hai\Desktop\Fix\Whom-5-CNPM\backend\app\rag\tutor_service.py:3: FutureWarning:
All support for the 'google.generativeai' package has ended. It will no longer be receiving updates or bug fixes. Please switch to the 'google.genai' package as soon as possible.
See README for more details:
https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md
  import google.generativeai as genai
tests/test_analytics_progress.py: 5 warnings
tests/test_parent_api.py: 9 warnings
tests/test_practice.py: 5 warnings
tests/test_study_plan_current.py: 4 warnings
tests/test_teacher_and_rag.py: 4 warnings
tests/test_teacher_api.py: 3 warnings
tests/test_tutor_api.py: 1 warning
C:\Users\Duc Hai\Desktop\Fix\Whom-5-CNPM\venv\Lib\site-packages\sqlalchemy\sql\schem.py:3627: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  return util.wrap_callable(lambda ctx: fn(), fn) # type: ignore
tests/test_practice.py: test_submit_practice_scoring_and_persistence
C:\Users\Duc Hai\Desktop\Fix\Whom-5-CNPM\backend\app\practice_routes\practice.py:188: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  completed_at=datetime.utcnow(),
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
56 passed, 34 warnings in 8.06s
```

Figure 27: Automated Pytest Execution Results across 13 Test Suites (56/56 Passed)

### Verified Modules & Test Suites Summary

1. **Integration & Auth Suite** (tests/test\_integration.py - Passed 8/8 tests): Validates user schema mapping, student registration, duplicate email handling (HTTP 409 Conflict), login token generation, user profile inspection (/me), refresh token rotation, inactive user access locking, and full 4-role RBAC authorization matrix enforcement.

2. **JWT Contract Suite** (tests/test\_jwt\_contract.py - Passed 4/4 tests): Verifies access token claims (sub, role, type, jti, exp), refresh token lifespan integrity, rejection of expired cryptographic signatures, and Bcrypt password verification.
3. **RAG Pipeline Suite** (tests/test\_rag.py - Passed 3/3 tests): Tests text chunking sliding window algorithm (chunk size = 100, overlap = 20), L2 vector normalization (magnitude = 1.0), and Cosine Similarity Top-K semantic retrieval.
4. **Standalone RBAC Suite** (tests/test\_rbac\_standalone.py - Passed 7/7 tests): Validates isolated dependency injection permission guards and verifies HTTP 403 Forbidden enforcement on unauthorized role requests.
5. **SDD Compliance Suite** (tests/test\_sdd.py - Passed 1/1 test): Validates architectural documentation and API contract consistency against the Software Design Document.
6. **Study Plan & Goal Suite** (tests/test\_study\_plan\_target\_score.py - Passed 4/4 tests): Verifies database NUMERIC(7,2) precision for target scores up to 1200 points and enforces business validation constraints on out-of-range inputs.
7. **Practice Module Suite** (tests/test\_practice.py - Passed 5/5 tests): Tests practice question querying with subject/topic/difficulty filters, STUDENT role enforcement, automated score calculation, test persistence, and completed practice history retrieval.
8. **AI Tutor API Suite** (tests/test\_tutor\_api.py - Passed 4/4 tests): Verifies HTTP 401 on unauthenticated access, HTTP 403 on invalid roles (PARENT), HTTP 404 on missing questions, and successful context-aware AI tutor response generation with mocked Gemini API calls.
9. **Active Study Plan Suite** (tests/test\_study\_plan\_current.py - Passed 3/3 tests): Validates retrieval of the student's currently active study roadmap (/api/study-plan/current), returns structured daily tasks (PlanTask), and enforces HTTP 404 handling when no active roadmap exists.
10. **Student Analytics & Progress Suite** (tests/test\_analytics\_progress.py - Passed 3/3 tests): Verifies competency accuracy calculations broken down by academic subjects (Toán học, Logic, Tiếng Việt), historical daily score tracking, and predicted exam score range generation ( $S_{\text{predict}} \in [1, 1200]$ ).
11. **Teacher Management API Suite** (tests/test\_teacher\_api.py - Passed 4/4 tests): Validates teacher-exclusive RBAC permissions (rejecting student access with HTTP 403), full CRUD question bank operations (Create, Read with pagination/filters, Update, Delete), and system-wide student performance analytics overview.
12. **Teacher Overview & RAG Retrieval Suite** (tests/test\_teacher\_and\_rag.py - Passed 6/6 tests): Validates the GET /api/teacher/overview endpoint for aggregate question counts, student counts, and attempt analytics; enforces RBAC restriction (HTTP 403 for students).

on teacher overview); validates the semantic RAG pipeline (/api/v1/rag/query) for both TEACHER and STUDENT roles (HTTP 200 OK); and verifies input validations on empty query/document payloads.

13. **Parent Portal API Suite** (tests/test\_parent\_api.py - Passed 4/4 tests): Validates unauthenticated request rejection (HTTP 401 Unauthorized); verifies RBAC enforcement against student access (HTTP 403 Forbidden); handles edge cases for parent accounts without linked children; and verifies successful data aggregation on GET /api/parent/overview and GET /api/parent/progress including subject accuracy radar metrics, daily exam scores, and score predictions.